IJRCS
International Journal of Robotics and Control Systems
ISSN 2775-2658
http://pubs2.ascee.org/index.php/ijrcs
ASCEE

A Controlled Multi-Seed Evaluation of Relational Knowledge Distillation for Motor Imagery Decoding with Classical and Quantum Kernels

Duong Thi Mai Thuong [a,1], Nguyen Phuong Huy [b,2,*]

[a] Department of Computer Science, Faculty of Information Technology, Thai Nguyen University of Information and Communication Technology, Thai Nguyen, Vietnam
[b] Department of Electronics Engineering, Faculty of Electronics Engineering, Thai Nguyen University of Technology, Thai Nguyen, Vietnam
Email: [1]dtmthuong@ictu.edu.vn, [2]nguyenphuonghuy@tnut.edu.vn
[*] Corresponding author: nguyenphuonghuy@tnut.edu.vn
[1]ORCID: 0009-0005-7862-7473; [2]ORCID: 0000-0002-5887-5796

ARTICLE INFO

Article History
Received
Revised
Accepted

Keywords
Brain–computer interface;
Knowledge distillation;
Motor imagery decoding;
Quantum kernel;
Reproducibility

ABSTRACT

Knowledge distillation can transfer the accuracy of large motor imagery decoders to compact networks for embedded brain–computer interfaces. However, distillation mechanisms are usually compared from a single training run, so their rankings may reflect training noise. The research contribution is a controlled multi-seed evaluation protocol for these mechanisms, together with quantitative evidence that seed noise exceeds the differences among them. A structured relational distillation framework varies the relation-formation strategy and the similarity kernel, classical or quantum, with all other components fixed. Twelve mechanisms were evaluated on PhysionetMI (103 subjects) with subject-wise cross-validation, five fixed seeds, and a statistical plan fixed before running. In this confirmatory analysis, distillation with a 30-epoch budget raised accuracy from 74.93% to 81.49%, and an exploratory replication on Cho2017 (49 subjects) agreed in direction. Beyond this gain, no relation strategy or kernel separated significantly from logit-level distillation. The spread among mechanisms (0.46 percentage points) was smaller than seed noise (0.75 points), and single-run rankings were almost uncorrelated with five-seed rankings (Spearman $\rho = 0.14$). The quantum kernel did not differ significantly from cosine or RBF kernels, even with gradient-matched weighting, but these data cannot establish equivalence. A factorial experiment crossing the two design axes detected no interaction between them. The quantum branch exists only during training, so the deployed student runs unchanged on a Raspberry Pi 4, at about 13 times the training cost. Mechanism-level claims in electroencephalography distillation should therefore rest on multi-seed comparisons with confidence intervals.

© 2026 The Authors.
Published by Association for Scientific Computing Electrical and Engineering.
This is an open access article under the CC-BY-SA license.

1. Introduction

A brain–computer interface (BCI) translates brain activity into control signals for external devices without passing through the usual neuromuscular pathways [1–3]. Its applications have spread from

the laboratory to practical systems [4, 5] such as powered wheelchairs [6–8], assistive robotic arms and prostheses [9], upper-limb rehabilitation devices [10], and intention-driven vehicles [11]. This study focuses on motor imagery (MI), a paradigm suited to sustained, self-paced control because it needs no external stimulus [6]. In these systems, the EEG decoder is one stage of a closed control loop. Its inference time must fit within the time budget of the loop [12, 13]. The model must run on the embedded hardware built into the device, where time, memory, and energy must be considered together [14–19]. Its reliability must remain stable across sessions and subjects, especially when the BCI shares control with an autonomous system [20–22]. The move toward consumer devices with few channels further degrades signal quality [23–25]. The paper uses the information transfer rate (ITR) [26] and the closed-loop time budget as evaluation criteria alongside accuracy (Section 2.6).

These requirements confront the designer of a motor imagery decoder with a conflict between accuracy and deployability. Deep learning has substantially improved motor imagery decoding over hand-crafted features [27, 28]. Compact architectures such as EEGNet [29] and its variants [30–33] can be deployed on embedded hardware. Higher-capacity models range from deep convolutional networks [34], transformers, and hybrid architectures [35–39] to multi-component ensembles [40]. They are more accurate but costly at inference and strongly dependent on the amount of training data [41, 42]. Knowledge distillation (KD) targets this conflict directly by transferring knowledge from a large teacher to a small student without increasing the inference cost of the student [43]. KD is now one of the main strategies for building compact EEG decoders [44, 45], alongside pruning, quantization, and hardware acceleration [46].

Two gaps limit the value of existing comparisons of distillation mechanisms for EEG. The first gap concerns how distillation mechanisms are compared with one another. Training deep networks is affected by several sources of randomness, and this effect can be larger on EEG data, which vary strongly both between and within subjects [47]. Changing the random seed alone can produce substantial differences between training runs of the same architecture [48]. Comparisons of machine learning methods need to account for these sources of variance [49], and similar concerns have been raised for deep learning on neural data [50]. Even so, comparisons among distillation mechanisms for EEG are still commonly reported from a single run. The extent to which a single-run ranking reflects real differences has hardly been measured directly. The second gap lies in the design of relational distillation mechanisms themselves. In existing work, the space in which relations are measured and the way sample pairs are chosen always change together. It is therefore impossible to tell whether an observed effect comes from the representation space or from how relations are organized.

Distillation methods are grouped by the form of transferred knowledge into output-based, feature-based, and relation-based methods [51, 52]. Feature-based methods use hints from hidden layers [53] and have been applied to biomedical images [54]. Relation-based methods represent knowledge through distances and angles between representation vectors [55], pairwise similarity [56], local neighborhood relations [57], or recursively constructed relations [58]. These groups have been extended to self-distillation [59, 60] and to physiological signals [61–65]. For EEG, relations among samples appear particularly suitable because discriminative information often lies in the relative structure among samples. Similarity-Keeping Knowledge Distillation preserves the similarity structure when moving to a system with fewer electrodes [66]. Multi-level distillation frameworks have been proposed for motor imagery [67], and an ensemble teacher has been distilled into a single backbone under latency constraints [68]. In these works, the sample pairs used to build relations are mostly chosen empirically, and relations are always measured in a classical metric space.

Quantum machine learning offers a fundamentally different space for measuring relations. A parameterized quantum circuit maps data into a Hilbert space, where the inner product between two states acts as a kernel function [69–71]. A kernel that is hard to compute classically does not by itself imply a learning advantage [72]. The performance of a quantum kernel also depends strongly on its

feature map and hyperparameters [73]. Exploiting such kernels is still limited by the qubit count, circuit depth, and noise of current hardware [74, 75]. In biomedicine and EEG, the quantum component is usually inserted into the backbone or placed at the end of the processing chain [76–80]. Designs in other fields follow the same pattern [81–83].

Quantum knowledge distillation has been used to compress quantum networks [84] and to synchronize distributed quantum models [85]. It has also transferred knowledge from a classical teacher to a quantum student [86, 87] and let a classical network emulate quantum dynamics [88]. In the compression and quantum-student directions, the quantum component remains in the inference model. As in the EEG designs above, the deployed model must then access quantum hardware or a simulator at run time, which conflicts with the latency and resource constraints. Quantum Relational Knowledge Distillation (QRKD) and Quantum-Train use the quantum circuit only during training [89, 90]. Their premise is that quantum kernels describe relations among representations differently from classical kernels [91]. This approach suits a decoder in a control loop, but QRKD has only been verified on vision and language with a fixed relation structure.

Table 1 compares this study with the closest relational distillation works by data, relation space, pair selection, and use of quantum components at inference.

Table 1. Comparison of this study with the closest relational distillation works.

Work	Data	Relation space	Pair selection	Quantum at inference
SKD [66]	EEG, high- to low-electrode density	Euclidean (features)	Fixed	No
Multi-level distillation [67]	Motor imagery EEG	Classical	Not a surveyed factor	No
QRKD [89]	Vision, language	Hilbert (quantum kernel)	Fixed	No
This study	Motor imagery EEG (PhysionetMI, Cho2017)	Cosine, RBF, quantum kernel	Five strategies, a surveyed factor	No

This study builds a structured relational distillation framework (QRD) to address both gaps under tightly controlled conditions. The QRD framework treats the relation-formation strategy and the space used to measure similarity as two separate design axes and varies one axis at a time. The interaction between the two axes is tested separately with a factorial experiment. Five relation strategies are examined, ranging from dense relations and sparse relations based on representative samples to selection guided by the uncertainty of the teacher. They are combined with cosine, RBF, and quantum kernels on the same loss function, with every other component kept fixed. The quantum kernel is not included as a proposed solution. It is the strictest test case, because the geometry of Hilbert space cannot be reduced to Euclidean distance in the original feature space. Every relation kernel participates only in computing the loss, so the model deployed into the control loop remains a compact classical network.

The research contribution is a controlled evaluation protocol for distillation mechanisms on motor imagery EEG, together with quantitative evidence that random-seed noise exceeds the differences among mechanisms. It comprises the following three elements, in order of importance.

- The main contribution is the evidence of seed instability. It was obtained with a protocol that uses the subject as the statistical unit, five fixed seeds, and a statistical plan fixed before running. Seed noise (about 0.75 percentage points) is larger than the differences among mechanisms (about 0.46 points). The ranking from a single run is almost uncorrelated with the ranking over five seeds (Spearman coefficient 0.14).

- The QRD framework is the controlled test bed for this evidence. Within it, distillation provides a stable benefit on PhysionetMI and Cho2017. However, no relation variant or kernel, including the quantum kernel, separates from the KDBase baseline at statistical significance.
- Every configuration is additionally evaluated from a control-oriented perspective, using ITR, the inference-time budget, and a real inference-time measurement on embedded hardware.

The remainder of the paper is organized as follows. Section 2 presents the research workflow, the QRD framework with its variants, and the evaluation protocol. Section 3 presents and discusses the results, and Section 4 concludes and outlines directions for future development. Implementation details, additional controls, and full result tables are given in the appendices.

2. Method

2.1. Research workflow and overview of the QRD framework

Figure 1 summarizes the research workflow, from data to statistical conclusions. The signals are first preprocessed and the 103 PhysionetMI subjects are divided into five folds by a deterministic rule that does not depend on the seed (Section 2.5). The teacher is then trained once per fold and kept fixed. For each combination of seed, fold, and distillation configuration, the student is trained and its checkpoint is selected by validation accuracy. The per-subject results are aggregated into one value per seed before testing according to the pre-specified plan. Finally, the same workflow is replicated exploratorily on Cho2017, and the inference time of the student is measured on a Raspberry Pi 4. Section 2.9 presents this workflow as pseudocode (Algorithm 2). The rest of this part focuses on the QRD framework, that is, the configuration block c and the QRD branch in Figure 1.

This part describes how the components of the QRD framework connect to one another and defines the role of each component during training. Figure 2 shows the entire architecture.

The framework is organized as a teacher–student setup. The teacher is a pretrained ensemble model [40], kept fixed and serving only as the source of the supervisory signal. The student is an EEGNet network [29] and is the only component in the teacher–student pair whose parameters are updated [43]. The exception concerning the quantum circuit shared between the two branches is discussed in Section 2.3.

This pair of models was chosen to concretely realize the conflict noted above. The ensemble model aggregates information from multiple components and therefore provides a stable supervisory signal. However, its memory and computational cost increases with the number of components, making it poorly suited to the embedded hardware built into the device. EEGNet, conversely, has a parameter count and inference time suited to embedded hardware and has been verified in practically deployed BCI systems [17]. The performance gap between the two models is precisely the quantity that the distillation framework seeks to narrow.

The supervisory signal delivered to the student consists of a base component that matches the soft prediction distribution at the logits level. Depending on the configuration, at most one of three additional components is added. The general loss function for this combination is written in (6), with the coefficients of the components not belonging to the configuration under consideration set to zero. The first two additional components belong to classical distillation, comprising alignment of intermediate features and preservation of the geometric structure among samples in feature space. Both are presented in Section 2.2 and used as the basis for comparison.

The third additional component is the quantum relation branch, the component that this study adds. This branch takes the intermediate features of both models, constructs a relation matrix in Hilbert space, and produces an additional loss component. It exists only in the computational graph of the

training stage, does not appear in the inference path, and therefore does not change the configuration of the deployed model.

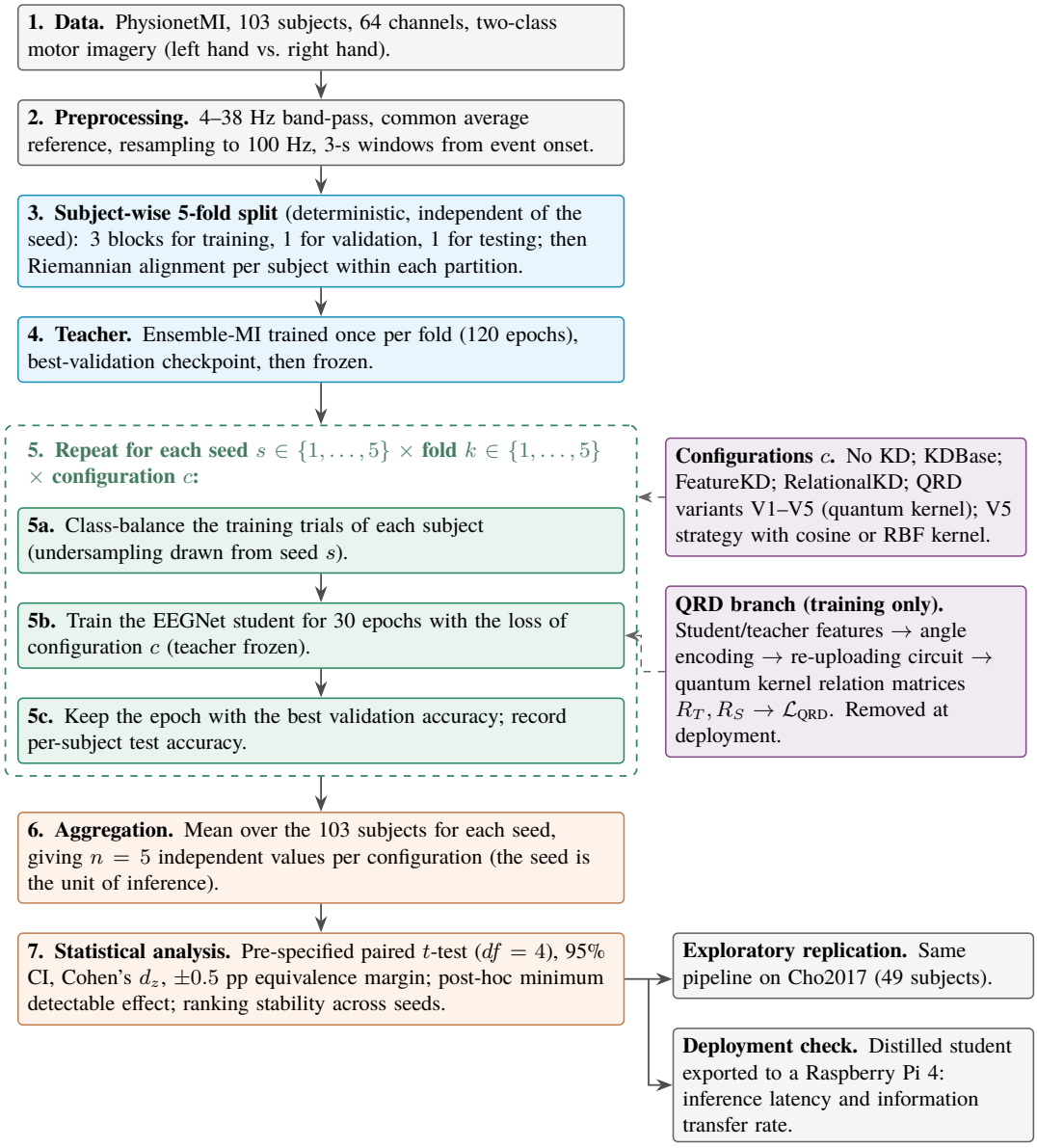


Fig. 1. Research workflow, from data to statistical analysis. The dashed frame is the loop over seeds, folds, and configurations. The QRD branch exists only during training.

The remainder of this part establishes the notation used consistently throughout Section 2. Consider a mini-batch of N EEG samples $\{x_i\}_{i=1}^{N}$. For each sample, each model produces two types of output. These are the logit vectors $z_T(x_i), z_S(x_i) \in \mathbb{R}^C$ at the classification layer and the feature vectors $f_T(x_i), f_S(x_i) \in \mathbb{R}^d$ at the last representation layer before it. For the pair of models used, $C = 2$ and $d = 144$ for both the teacher and the student.

Stacking the feature vectors by column gives the feature matrix of the mini-batch, matching the convention used in Figure 2:

$$F_T = \begin{bmatrix} f_T(x_1) & \cdots & f_T(x_N) \end{bmatrix} \in \mathbb{R}^{d\times N}, \qquad F_S = \begin{bmatrix} f_S(x_1) & \cdots & f_S(x_N) \end{bmatrix} \in \mathbb{R}^{d\times N}. \tag{1}$$

Distillation at the relational level does not compare F_T with F_S directly but compares the relation

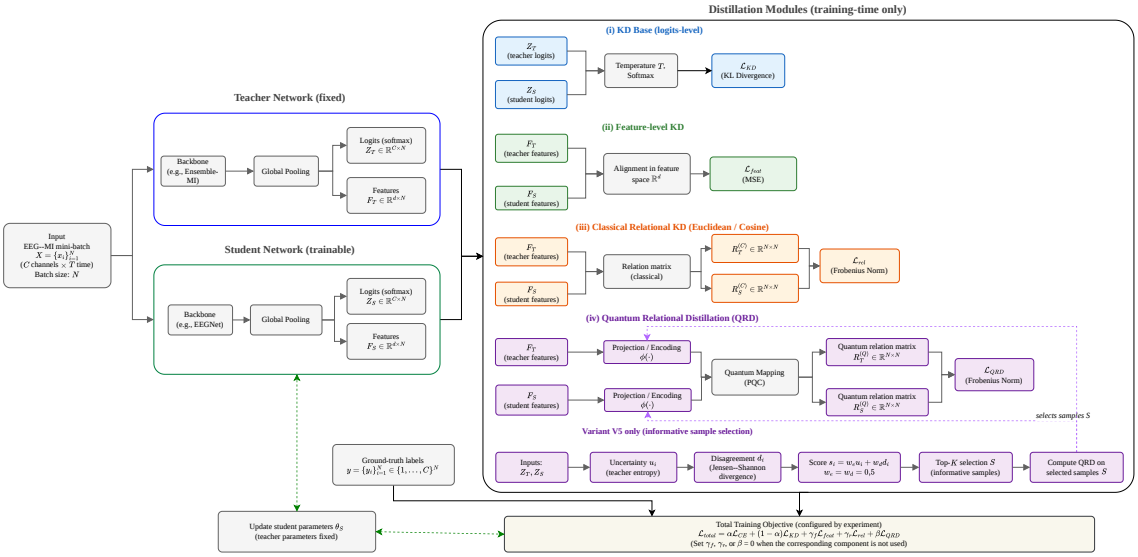


Fig. 2. Overall teacher–student framework with the proposed quantum relational distillation branch.

structure among samples within each model. This structure is represented by two symmetric matrices R_T and R_S. Element (i, j) of each matrix measures the similarity between samples x_i and x_j according to the representation of the corresponding model. In classical relational distillation methods, these two matrices are constructed from Euclidean distance or cosine similarity in feature space [55]. Section 2.3 replaces that space with a Hilbert space, in which similarity is measured by a quantum kernel function.

The relation matrix is not necessarily computed over the entire mini-batch. The set of participating index pairs, denoted $\mathcal{I}$, is a configuration parameter and is defined separately for each variant in Section 2.4.

2.2. Classical distillation setups

This part defines three classical distillation configurations used as baselines. All three share the same teacher, student, dataset, and optimization protocol. The only component that changes is the form of the additional supervisory signal. Because of this constraint, any observed performance difference can be attributed to the knowledge-transfer mechanism.

KDBase performs distillation at the output level, matching the student's soft prediction distribution to the teacher's through temperature-softened Kullback–Leibler divergence [43]:

$$\mathcal{L}_{\mathrm{KD}} = \tau^2 \,\mathrm{KL}\big(\sigma\big(\tfrac{z_T}{\tau}\big) \,\big\|\, \sigma\big(\tfrac{z_S}{\tau}\big)\big)\,, \tag{2}$$

where $\sigma(\cdot)$ is the softmax function and $\tau = 4.0$ is the softening temperature. The coefficient τ^2 keeps the gradient magnitude independent of τ.

FeatureKD adds a constraint at the intermediate feature level [53]. Because the student and teacher have the same feature dimensionality, the constraint is applied directly without an intermediate projection layer:

$$\mathcal{L}_{\mathrm{feat}} = \frac{1}{Nd}\,\|F_S - F_T\|_F^2\,. \tag{3}$$

RelationalKD shifts the focus from individual samples to relations among samples [55]. This study uses the pairwise-distance component of the original method. For each model, the pairwise

Euclidean distance matrix is normalized by the mean of its off-diagonal elements:

$$\tilde{D}_{ij} = \frac{\|f_i - f_j\|_2}{\frac{1}{N(N-1)} \sum_{p \neq q} \|f_p - f_q\|_2}, \tag{4}$$

and the discrepancy between the two normalized matrices is measured by the Huber function:

$$\mathcal{L}_{\text{rel}} = \ell_\delta \left(\tilde{D}^S, \tilde{D}^T \right), \tag{5}$$

where ℓ_δ is the smooth L1 loss function. Normalizing by the mean makes the loss function invariant to uniform scaling of feature space. The student is therefore required to reproduce only the relative structure of the teacher's representation, not its absolute magnitude.

The total loss function of any configuration has the form

$$\mathcal{L} = \alpha \, \mathcal{L}_{\text{CE}} + (1 - \alpha) \, \mathcal{L}_{\text{KD}} + \gamma_f \, \mathcal{L}_{\text{feat}} + \gamma_r \, \mathcal{L}_{\text{rel}} + \beta \, \mathcal{L}_{\text{QRD}}, \tag{6}$$

where $\mathcal{L}_{\text{CE}}$ is the cross-entropy with the true labels, and the coefficients of the components not belonging to the configuration under consideration are set to zero. The component $\mathcal{L}_{\text{QRD}}$ is presented in Section 2.3. The coefficient values are given in Section 2.8.

The three configurations above form a sequence of increasing abstraction in the type of knowledge transferred, from output to individual features to relations among samples. RelationalKD is the first configuration in the sequence to represent knowledge in relational form, and it is also the point that the QRD framework extends. Instead of measuring relations by Euclidean distance in feature space, QRD measures them by a kernel function in Hilbert space.

2.3. Quantum relational distillation framework

The quantum kernel function is included to extend the scope of the conclusions rather than as a proposed solution (Section 1). The classical relation strategies all construct structure within the same metric space. If only these were compared with one another, a negative result could be attributed to the representation space not actually having been changed.

The quantum relation branch consists of three steps: dimensionality reduction and encoding of features into rotation angles, quantum state preparation, and kernel computation to construct the relation matrix.

Dimensionality reduction and angle encoding. The $d = 144$-dimensional feature vector cannot be encoded directly into an $n_q = 4$-qubit circuit. Each model is assigned its own linear projection $W_T, W_S : \mathbb{R}^d \to \mathbb{R}^{n_q}$. The teacher-side projection W_T is initialized and kept fixed throughout training. Only the student-side projection W_S is the trainable parameter among these two projections. Variant V1 in Section 2.4 replaces W_S with a deterministic dimensionality reduction, in which case the student-side quantum branch also has no trainable parameter at the projection step.

The projection W_T is initialized within the same global random number stream, so it is likely to differ across seeds. The effect of this variation is checked with a control that fixes W_T (Appendix A). Initialization details and the theoretical limits of the projection are given in Appendix B. The output $Wf(x_i)$ of the projection is not bounded in magnitude, whereas the rotation angle used for the quantum gate needs to lie within a defined range. Therefore, this output is bounded into the valid angle range using the hyperbolic tangent function before being used as the rotation angle:

$$a(x_i) = \pi \, \tanh\big(Wf(x_i)\big) \in [-\pi, \pi]^{n_q}. \tag{7}$$

This bounding limits the rotation angle to a full period, reducing the risk that two different feature values map to the same state due to the periodicity of rotation.

State preparation. The angle vector is loaded into the circuit via angle encoding around the Y axis, denoted $\Phi(\cdot)$. The encoding alternates with $L = 2$ Strongly Entangling Layers of entangling gates [70], denoted $U(\theta_l)$ for the l-th layer. Unlike a single-pass encode-then-entangle circuit, the encoding is repeated, alternating with the entangling layers, in a *data re-uploading* style [92]:

$$|\varphi_l(x_i)\rangle = U(\theta_l)\,\Phi\big(a(x_i)\big)\;|\varphi_{l-1}(x_i)\rangle\,, \qquad |\varphi_0(x_i)\rangle = |0\rangle^{\otimes n_q}\,, \qquad l = 1,\ldots,L, \tag{8}$$

$$|\psi(x_i)\rangle = \Phi\big(a(x_i)\big)\;|\varphi_L(x_i)\rangle\,. \tag{9}$$

For $L = 2$, the full gate sequence is encode–entangle–encode–entangle–encode. The data is re-loaded three times, alternating with two parametrized entangling layers. The parameters $\theta = (\theta_1,\ldots,\theta_L)$ are initialized once from the normal distribution $\mathcal{N}(0, 0.1)$. They are trainable parameters, updated via backpropagation together with W_S throughout training. The circuit $U(\theta_l)$ uses a single shared set θ for both the teacher branch and the student branch, without separating θ_T/θ_S. The consequence of this choice for the teacher's "fixed" status is analyzed later in this part.

Training θ is only meaningful if the entangling layer $U(\theta_l)$ actually affects the kernel function rather than canceling itself out of the computation. This risk is real in a single-pass encode-then-entangle circuit, $|\psi(x_i)\rangle = U_\theta\,\Phi(a(x_i))\;|0\rangle^{\otimes n_q}$, where the entangling layer U_θ is an operator that is *identical for every sample* and lies outermost in the expression. The kernel then depends only on the inner product $\langle\psi(x_i)|\psi(x_j)\rangle = \langle 0|\,\Phi(a(x_i))^\dagger U_\theta^\dagger U_\theta \Phi(a(x_j))\,|0\rangle$. Because $U_\theta^\dagger U_\theta = I$ for every θ, the entangling layer cancels out of the kernel regardless of how it is trained, and the circuit degenerates into a classical kernel function of Φ alone.

The data re-uploading structure in (8)–(9) avoids this degeneracy. The final encoding $\Phi(a(x_i))$ is a *sample-dependent* operator and lies outermost instead of U_θ, so the same reduction no longer applies. $U(\theta_l)$ cannot be removed from the inner product by a data-independent unitarity argument. Empirical verification on a fixed batch gives a difference of 0.474 between the quantum kernel and the corresponding cosine product (Appendix B). The circuit therefore does not degenerate into a classical kernel.

The circuit is then no longer a fixed random feature map into the 2^{n_q}-dimensional Hilbert space as in a single-pass encoding design, but a *trainable* feature map. The data-adaptive capacity of the quantum branch lies in W_S and θ. W_T remains fixed as noted above.

Kernel function and relation matrix. The similarity between two samples is measured by the fidelity between their corresponding states [69]:

$$K\big(a_i, a_j\big) = \big|\,\langle\psi(x_i)|\psi(x_j)\rangle\,\big|^2 \in [0,1]. \tag{10}$$

Applying this separately to each model gives two quantum relation matrices:

$$R_T^{(Q)}(i,j) = K\big(a_T(x_i), a_T(x_j)\big), \qquad R_S^{(Q)}(i,j) = K\big(a_S(x_i), a_S(x_j)\big), \tag{11}$$

where a_T and a_S are the angle vectors obtained from (7) with the corresponding projection. The two matrices are used directly without any additional normalization step, because the kernel already takes values in $[0,1]$ by definition.

Loss function. The discrepancy between the two relation structures is measured by the mean squared error over the index-pair set $\mathcal{I}$:

$$\mathcal{L}_{\text{QRD}} = \frac{1}{|\mathcal{I}|}\sum_{(i,j)\in\mathcal{I}}\Big(R_T^{(Q)}(i,j) - R_S^{(Q)}(i,j)\Big)^2. \tag{12}$$

The gradient is propagated through the quantum circuit to the projection W_S and continues into the student network. This component therefore affects the student's entire learning process, not only

the projection layer. The projection W_T participates in computing $R_T^{(Q)}$ but does not receive gradients, as noted above.

However, $R_T^{(Q)}$ is not entirely static. The computation of $R_T^{(Q)}$ is not detached from the computational graph. No `detach` or `no_grad` step wraps the teacher branch in (12), so the gradient of $\mathcal{L}_{\text{QRD}}$ still propagates to θ through both the teacher branch and the student branch. The teacher's backbone and the projection W_T remain fixed, but the mapping from the teacher's features to $R_T^{(Q)}$ still changes with θ throughout training. In the classical distillation setups of Section 2.2, by contrast, the teacher target is entirely fixed.

In principle, sharing θ opens up an alternative optimization direction for (12). Training could shift θ to make the two matrices easier to match, for example by making both kernels flatter. The student would then no longer be forced to reproduce the exact relation structure that the teacher has learned. This is a different form of degeneracy from the classical-kernel degeneracy caused by the cancellation of U_θ examined above. Separating a fixed θ_T from a trainable θ_S would avoid this risk. In exchange, the two branches would no longer be measured by the same kernel function, which would undermine the basis for directly comparing $R_T^{(Q)}$ with $R_S^{(Q)}$ in (12). This study chooses to share θ and examines this trade-off with the control in Appendix A.

In the total objective function (6), the QRD configurations use $\gamma_f = \gamma_r = 0$. That is, the student simultaneously receives supervisory signal from the true labels, from the teacher's logits, and from the quantum relation structure. The coefficient β is two orders of magnitude smaller than the other coefficients. The reason is that $\mathcal{L}_{\text{QRD}}$ is a discrepancy between two quantities bounded in $[0, 1]$, so its magnitude is much smaller than that of the other loss components.

The same $\beta = 0.02$ is used for all three kernel function types compared in Section 2.4 (quantum, cosine, RBF). That all three kernels give values in $[0, 1]$ only guarantees that the scale of $\mathcal{L}_{\text{QRD}}$ lies within the same range. It does not guarantee that the three kernels produce the same gradient magnitude or the same influence on the student when multiplied by the same β. RBF self-normalizes according to the median distance of each batch, whereas the quantum kernel tends to concentrate values near the boundary because of the geometry of Hilbert space. More generally, the value distribution of a quantum kernel depends strongly on the embedding. Kernel values are known to concentrate as embeddings become more expressive [93]. Their effective scale is set by the encoding bandwidth [94]. The actual difference in supervisory magnitude among the three kernels is measured and addressed in Section 3.2.

2.4. Variants of the QRD framework

The five variants are built on the same quantum kernel function (10) and the same loss function (12). They differ in how the d-dimensional feature is reduced to n_q dimensions before quantum encoding. They also differ in the index set $\mathcal{I}$ that determines which sample pairs participate in the relation matrix. Figure 3 illustrates the five designs.

This design stems from the observation that not every sample pair carries the same amount of information. EEG signals have a low signal-to-noise ratio and vary strongly across subjects [47], so a dense relation matrix may contain many pairs that reflect noise more than class structure.

Variants V1 through V4 share one implementation point that needs to be stated first. The cost of simulating the quantum circuit grows quadratically with the number of samples. The relation matrix for these four variants is therefore computed not on the entire mini-batch of size N but on a *position-fixed* subsample. This subsample consists of the first $q = \min(16, N)$ elements of each batch. Because the batch has already been shuffled randomly, this subsample is equivalent to a content-independent random sampling. Every index set for V1 through V4 below is understood to be defined over those q samples.

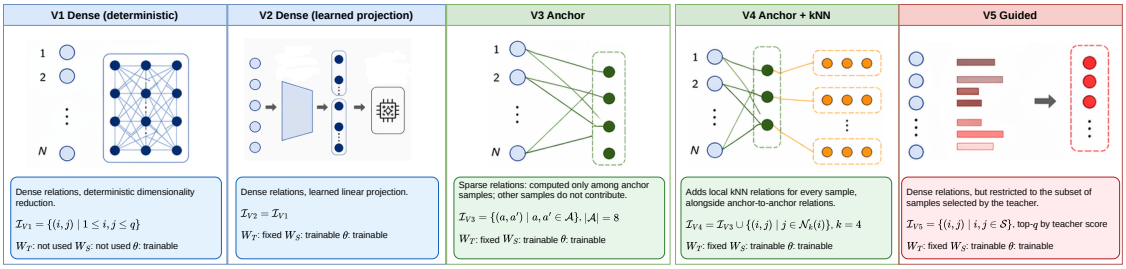


Fig. 3. The five variants of the QRD framework.

V1: dense relations, deterministic dimensionality reduction. This base variant does not use a learned linear projection at the dimensionality-reduction step. The quantum circuit θ is still trained as in every other variant (Section 2.3), so this variant has 24 additional parameters (only θ). The other four variants have 604 (580 from W_S plus 24 from θ). The d-dimensional feature vector is divided evenly into n_q consecutive groups, each averaged:

$$\big[g(f)\big]_m = \frac{n_q}{d} \sum_{l=(m-1)d/n_q+1}^{m\,d/n_q} f_l, \qquad m = 1, \ldots, n_q, \tag{13}$$

and then $g(f)$ replaces Wf in (7). The index set consists of all pairs:

$$\mathcal{I}_{\mathrm{V1}} = \{(i, j) \mid 1 \le i, j \le q\}. \tag{14}$$

This variant is the most direct extension of classical relational distillation into Hilbert space and serves as the comparison point for the other four variants.

V2: dense relations, learned projection. This variant keeps V1's index set but replaces the deterministic dimensionality reduction with the two linear projections W_T and W_S described in Section 2.3. Only W_S is a trainable parameter, while W_T remains fixed:

$$\mathcal{I}_{\mathrm{V2}} = \mathcal{I}_{\mathrm{V1}}. \tag{15}$$

The pair V1 and V2 compares two designs for projecting features into angle space. V1 uses a fixed, parameter-free averaging, whereas V2 uses a pair of linear projections of which only the student-side W_S is trained. This comparison changes more than the trainability of W_S alone. Moving from V1 to V2 also replaces the teacher's deterministic chunk-averaged target with a random (frozen) linear projection W_T. Instantiating the two additional linear layers in V2 also consumes extra random draws relative to V1's parameter-free averaging. The V1-versus-V2 comparison is therefore best read as a comparison between two overall dimensionality-reduction designs. It is not a controlled ablation that isolates the benefit or cost of a learnable W_S on its own. A causal claim specific to W_S would require a control that holds the teacher-side target representation and the random draws fixed between the two variants.

V3: sparse relations based on an anchor set. To reduce relation density, this variant restricts the index set to an anchor set $\mathcal{A}$ of size $M = 8$:

$$\mathcal{I}_{\mathrm{V3}} = \{(a, a') \mid a, a' \in \mathcal{A}\}, \qquad \mathcal{A} \subset \{1, \ldots, q\},\ |\mathcal{A}| = M. \tag{16}$$

The anchor set is the first M elements of the subsample. The sample order is reshuffled at every epoch, so this is a random selection not based on sample content. This choice is deliberate, isolating the effect of *reducing density* from the effect of *guided selection*, which is investigated in V5. Samples

outside $\mathcal{A}$ do not contribute to the relation loss component, although they still participate in the other components of (6).

V4: combining global and local relations. This variant adds neighborhood relations to the anchor block:

$$\mathcal{I}_{\mathrm{V4}} = \{(a, a') \mid a, a' \in \mathcal{A}\} \cup \{(i, j) \mid j \in \mathcal{N}_k(i)\}, \tag{17}$$

where $\mathcal{N}_k(i)$ is the set of the $k = 4$ nearest neighbors of sample x_i. Neighbors are determined by Euclidean distance on the teacher's features, that is, the local structure is taken from the already-trained model rather than from the student that is still learning. The resulting relation is symmetrized. If j belongs to the neighborhood of i, both pairs (i, j) and (j, i) belong to $\mathcal{I}_{\mathrm{V4}}$. Unlike V3, every sample in the subsample participates through its own neighborhood edges. Because $\mathcal{I}_{\mathrm{V4}}$ is defined as a set, a pair (i, j) that appears in both the anchor block and the kNN block is counted exactly once in (12), not accumulated.

V5: teacher-guided relation selection. The four variants above select samples independently of content, on a fixed subset of q elements. V5 replaces that fixed subset with a subset *selected by content*, with the candidate pool being the entire N samples of the batch rather than the first q elements. For each sample x_i in the entire batch ($i = 1, \ldots, N$), the teacher's uncertainty is measured by the normalized entropy of the prediction distribution:

$$u_i = -\frac{1}{\log C} \sum_{c=1}^{C} p_T(c \mid x_i) \log p_T(c \mid x_i), \tag{18}$$

where C is the number of classes, so that $u_i \in [0, 1]$. The degree of disagreement between the teacher and the student is measured by the Jensen–Shannon divergence:

$$d_i = \tfrac{1}{2}\mathrm{KL}(p_T \,\|\, m_i) + \tfrac{1}{2}\mathrm{KL}(p_S \,\|\, m_i)\,, \qquad m_i = \tfrac{1}{2}\,(p_T + p_S)\,, \tag{19}$$

where p_T and p_S are shorthand for $p_T(\cdot \mid x_i)$ and $p_S(\cdot \mid x_i)$. Compared with the L_2 norm, the Jensen–Shannon divergence has the advantage of being symmetric, bounded, and suited to comparing two probability distributions. The combined score is a weighted average of the two quantities, computed for all N samples:

$$s_i = w_e\, u_i + w_d\, d_i, \qquad w_e = w_d = 0.5, \qquad i = 1, \ldots, N. \tag{20}$$

The set $\mathcal{S}$ consists of the q samples with the highest scores selected from the entire N samples. The relation matrix is constructed on the pairs within that subset of size q:

$$\mathcal{I}_{\mathrm{V5}} = \{(i, j) \mid i, j \in \mathcal{S}\}, \qquad \mathcal{S} = \underset{i \in \{1,\ldots,N\}}{\arg\,\text{top-q}}\; s_i. \tag{21}$$

Unlike V1–V4, where the index set $\mathcal{I}$ is determined by a fixed position or the teacher's frozen features, the selection here is data-dependent. The set $\mathcal{S}$ depends on the score s_i through a discrete, non-differentiable selection. Meanwhile, the step of constructing the relation matrix on $\mathcal{S}$ still requires normal backpropagation. Algorithm 1 summarizes the computation sequence and clarifies the gradient boundary between these two parts.

Algorithm 1. Teacher-guided relation selection (V5).
Input: teacher logits $Z_T \in \mathbb{R}^{C \times N}$, student logits $Z_S \in \mathbb{R}^{C \times N}$, teacher/student features $F_T, F_S \in \mathbb{R}^{d \times N}$, selection set size q.
Output: the $q \times q$ quantum relation matrices used for $\mathcal{L}_{\mathrm{QRD}}$.

1: # Steps 1--3 lie outside the gradient graph (torch.no_grad)

2: **for** $i = 1, \ldots, N$: compute u_i via (18) from Z_T
3: **for** $i = 1, \ldots, N$: compute d_i via (19) from Z_T, Z_S
4: **for** $i = 1, \ldots, N$: $s_i \leftarrow w_e u_i + w_d d_i$ ((20))
5: $\mathcal{S} \leftarrow$ indices of the q largest values of s_i among the N samples (torch.topk, no backpropagation)
6: # From here, gradients are reconnected normally
7: $F_T[\mathcal{S}], F_S[\mathcal{S}] \leftarrow$ extract the q columns of F_T, F_S at indices $\mathcal{S}$
8: Encode $F_T[\mathcal{S}], F_S[\mathcal{S}]$ into quantum states via (7)--(9)
9: Construct $R_T^{(Q)}, R_S^{(Q)} \in \mathbb{R}^{q \times q}$ via (10)--(11) on the q selected samples
10: Compute $\mathcal{L}_{\text{QRD}}$ via (12) on these two $q \times q$ matrices

The candidate pool in lines 2–4 is the entire N samples, so the `top-q` operation in line 5 is a genuine selection that discards the $N - q$ lower-scoring samples. V1–V4, in contrast, perform no selection once the first q samples have been fixed by position. V5 therefore differs from V2–V4 in both the selection criterion and the candidate pool. Output-level information only guides sample selection, through a discrete rule fixed by (20). The relation matrix is still constructed from the intermediate features and fully participates in backpropagation (lines 6–10).

Table 2 summarizes the training status of each component across all five variants. It avoids confusing "no learned projection" (V1) with "no trainable parameters at all", which is not true for V1 because the circuit θ is still trained. The additional-parameters column does not count the teacher/student parameters that already exist outside the QRD branch.

Table 2. Training status of each component across the five QRD variants.

Variant	W_T	W_S	θ	Pair set $\mathcal{I}$	Additional parameters
V1 Dense (deterministic)	not used	not used	trained	full $q \times q$	24
V2 Dense (learned projection)	fixed	trained	trained	full $q \times q$	604
V3 Anchor	fixed	trained	trained	anchor $\mathcal{A}$, M=8	604
V4 Anchor+kNN	fixed	trained	trained	anchor + kNN, k=4	604
V5 Guided	fixed	trained	trained	teacher top-q	604

The five variants form two intersecting survey axes. The first axis, consisting of V1 and V2, examines the adaptability of the dimensionality reduction. The second axis, consisting of V2 through V5, examines the density and manner of relation selection. It moves from no intervention, to random density reduction, to adding local structure, and then to content-based sample selection. In parallel with these two axes, replacing the quantum kernel function with a cosine or RBF kernel function within the same strategy isolates the contribution of the representation space.

2.5. Data, preprocessing, and evaluation protocol

The method is evaluated on the PhysioNet EEG Motor Movement/Imagery dataset [95, 96], referred to as PhysionetMI after the name of its dataset class in MOABB. It contains EEG recordings from 109 subjects acquired with a 64-electrode system under the 10–10 standard at a sampling frequency of 160 Hz. Following the exclusion list of the teacher work [40], six subjects with recordings inconsistent in duration or label assignment were excluded (codes 88, 90, 92, 100, 104, and 106). The remaining 103 subjects were used.

The experiments focus on the two-class problem *left_hand* versus *right_hand*, the default two-class paradigm of the `PhysionetMI` class in MOABB. It distinguishes imagined left-hand and right-hand grasping, two basic control commands in brain-signal-operated assistive systems. The three remaining classes of the original dataset (rest, feet, hands) were not used. The number of trials per subject is not fixed, varying according to the actual number of recordings retained. For each subject, the *training* trials are undersampled per class to the smaller of the two class counts, by random sampling

without replacement drawn from the training seed. The training set is therefore class-balanced subject by subject. The random seed thus also controls which training trials are kept, in addition to weight initialization and batch order. The validation and test partitions keep every trial of the selected classes and are not rebalanced.

The raw 160-Hz signal is processed in four sequential steps. It is bandpass filtered at 4–38 Hz with a zero-phase FIR filter under the default MNE/braindecode configuration, without explicit order tuning. It is then re-referenced to the common average, resampled to 100 Hz, and windowed. There is no notch filtering step and no explicit artifact-removal step (no ICA, no trial-rejection threshold). Each epoch begins exactly at the event onset, with no start or end offset delay, and lasts 3.0 s, corresponding to 300 samples at 100 Hz after resampling.

After windowing and the train/validation/test split described below, a fifth step is applied: Riemannian covariance alignment, following the implementation of the teacher codebase [40]. The Riemannian geometry of covariance matrices is widely used in cross-subject motor imagery transfer to cope with inter-subject variability [97]. For every subject, and separately within each of the training, validation, and test partitions, a spatial covariance matrix is estimated per trial with the OAS estimator. These matrices are averaged across the trials of that subject using the Riemannian (affine-invariant) mean. The mean defines a whitening transform ($\mathrm{cov}^{-1/2}$), which is applied to every trial of that subject in that partition. This is the only step beyond unit conversion that touches the amplitude/covariance structure of the signal.

This alignment step has a direct consequence for how the reported accuracy should be interpreted. The whitening transform for a given test subject is estimated from that subject's own test-set trials, so the evaluation protocol is *offline and transductive at the subject level*. Aligning one test trial uses second-order statistics pooled over all of that subject's test trials. In an online single-command deployment, some of these trials would not yet be available at the time of inference. This step neither mixes information across subjects nor uses test *labels*. It is therefore not train/test leakage in the usual sense, in which preprocessing is fitted jointly on training and test data [98]. The accuracy figures in this paper quantify decoding when a batch of unlabeled data from the same session is available for alignment. They do not quantify single-window decoding for a new user with no prior data. Supporting an online or inductive deployment claim directly would require estimating this transform only from data available before each inference. Examples are a calibration block recorded at the start of a session or an online running estimate. Such a protocol has not been run in this study. The Pi 4 latency measurement in Section 3.4 times only the network's forward pass and does not include the cost of estimating or applying this alignment step.

The evaluation protocol is 5-fold cross-validation split by subject, to ensure that no subject appears simultaneously in the training set and the test set. This setup reflects real deployment conditions, where the system must operate with users who never participated in training.

Folds are split according to a deterministic rule. All 103 valid subjects, keeping their code order, are divided into 5 consecutive, nearly equal blocks. For each fold index, one block is chosen as the test set, the next block in cyclic order as the validation set, and the remaining three blocks as the training set. The corresponding split sizes are approximately 62 subjects for training, 21 for validation, and 20 for testing. The validation set is always completely separated by subject from both other sets. This split does not depend on the random seed. The same fold index always yields the same set of subjects across all five seeds. The seed governs only the randomness of training (weight initialization, batch order, and class-balancing undersampling), not how the data is split. The complete list of 103 subject-to-fold assignments is provided with the published source code but is not printed in the paper because of its length.

The entire procedure is repeated with five independent random seeds. The accuracy of each configuration is computed with the subject, not the trial or the fold, as the statistical unit. This avoids the

overly optimistic evaluation caused by the high correlation among samples within the same subject. Each subject enters the test set exactly once across the 5 folds, so each configuration is aggregated over $5 \times 103 = 515$ per-subject results.

These results are not independent of one another, because subjects within the same fold of the same seed share a single student model. The unit of inference for hypothesis tests is therefore the seed. Each seed reduces the 515 results to a single mean over subjects. Every paired t-test in the paper uses $n = 5$ independent pairs, with $df = 4$ degrees of freedom. The "mean $\pm$ standard deviation" values in the tables are likewise computed over these five per-seed values. Alongside p values, the paper reports differences with 95% confidence intervals and Cohen's d_z of the paired difference. This choice of unit confirms whether a conclusion is robust to the randomization of training. It does not confirm generalization to a different subject population, because every seed is evaluated on the same 103 subjects. The generalization question is examined separately through an exploratory replication on Cho2017. The statistical plan for the main kernel comparison, including the number of seeds, the test, and the ± 0.5 percentage-point equivalence margin, was recorded in an internal analysis plan written before running. This plan was not registered on a public registry and is released with the code. The ± 0.5-point margin was not derived from a power calculation. Examined after running, it corresponds to about 0.2 bit/min of ITR at the observed accuracy level (from 5.97 to 6.40 bit/min around 6.18 bit/min, see Section 2.6). It also exceeds the largest deviation between two reruns of the same configuration (0.37 points, Section 3.3 and Appendix C).

2.6. Control-oriented evaluation criteria

Besides classification accuracy, the paper evaluates configurations using quantities of the control loop. Information transfer rate measures the bandwidth of the control channel [26]. For an N-class problem with accuracy P, the amount of information obtained per command is

$$B = \log_2 N + P \log_2 P + (1 - P) \log_2 \left(\frac{1 - P}{N - 1} \right), \tag{22}$$

and the information transfer rate in bits per minute is

$$\mathrm{ITR} = B \times \frac{60}{T}, \tag{23}$$

where T is the time required for one command, in seconds. For the two-class case, expression (22) reduces to $B = 1 - H(P)$, where H is the binary entropy function. In the mid-accuracy range, a small improvement in accuracy produces a bandwidth gain much larger than the corresponding ratio. ITR computed via (23) relies on several assumptions about operating conditions and can give biased estimates if applied carelessly in online systems [99].

The time T does not consist solely of the signal window length. The time budget of the closed loop is decomposed as

$$T_{\text{loop}} = T_{\text{window}} + T_{\text{infer}} + T_{\text{command}}, \tag{24}$$

where T_{window} is the length of the signal segment to be acquired and T_{infer} is the model's inference time. The last term, T_{command}, is the time to transmit and execute the command on the actuator. Model design directly affects only T_{infer}, which is also the term most often neglected in offline evaluation studies.

Cohen's kappa coefficient is often preferred in BCI evaluation because it corrects for chance agreement. This advantage only takes effect when the classes are imbalanced or number more than two. For a two-class problem with exactly equal class counts, kappa is a deterministic linear transformation of accuracy ($\kappa = 2\,\mathrm{acc} - 1$) and carries no additional information. The two classes here are only approximately balanced in the test set (Section 2.5), so this identity holds approximately. For this reason, kappa is not reported separately.

2.7. Teacher and student network architectures

The teacher–student pair was chosen to maximize the separation between representational capacity and deployability. The teacher is Ensemble-MI [40], a classification system consisting of multiple convolutional branches operating in parallel. It is trained using a curriculum and collaborative learning mechanism. With about 15.1 thousand parameters, this model is about 6.3 times larger than the student. Its inference is 7.1 times slower because computational cost scales with the number of branches. The teacher is trained separately for each fold, yielding 5 independent checkpoints corresponding exactly to the 5 data splits in Section 2.5. Each checkpoint is trained for 120 epochs with 7 collaborative-learning subsets, following the original Ensemble-MI procedure exactly.

The student is EEGNet [29], using depthwise and separable convolutions to sharply reduce the number of parameters. With about 2.4 thousand parameters (2,402, about 10.2 million MACs per inference, see Section 3), this architecture has the potential to be deployed on a microcontroller. This parameter scale is in the same class as a published EEGNet deployment on embedded hardware [17]. It has not, however, been verified directly on that class of ultra-low-power microcontroller. Section 3.4 reports a direct inference-time measurement on real embedded hardware (Raspberry Pi 4). Both models produce intermediate feature vectors of the same dimensionality $d = 144$, so the feature-level and relation-level loss functions are applied directly without an intermediate projection layer.

The teacher is pretrained, kept fixed, and has gradient updates disabled throughout the entire distillation process. On this dataset, the teacher achieves an accuracy of $84.98\% \pm 2.59\%$ (mean and standard deviation over 5 folds).

2.8. Experimental configuration and hyperparameters

All experiments are implemented using PyTorch combined with PennyLane, running on a single GPU of a high-performance computing system. Table 3 summarizes the full configuration.

Every distillation configuration shares the same teacher, student, dataset, fold split, seed set, and

Table 3. Experimental configuration and hyperparameters.

Data and protocol		*Loss weights*	
Dataset	PhysionetMI	α (cross-entropy)	0.5
Number of subjects	103 out of 109	τ (temperature)	4.0
Channels, sampling rate	64, 160 Hz	γ_f (FeatureKD)	1.0
Task	Two-class	γ_r (RelationalKD)	1.0
Cross-validation	5-fold by subject	β (QRD)	0.02
Number of seeds	5		
Statistical unit	Subject	*Quantum and relation selection*	
Model		Number of qubits n_q	4
Teacher	Ensemble-MI, 15.1 K	Number of entangling layers L	2
Student	EEGNet, 2.4 K	Circuit weights	trainable
Feature dimension d	144	Subsample size q	16
		Number of anchor samples M	8
Optimization		Number of neighbors k	4
Optimizer	SGD, momentum 0.9	w_e, w_d	0.5 and 0.5
Learning rate	0.005		
Weight decay	0.01		
Batch size	64		
Number of epochs	30		
Checkpoint selection	by validation accuracy		
Library	PyTorch, PennyLane		

all optimization hyperparameters. The only component that changes across configurations is the knowledge-transfer mechanism. Because of this constraint, any observed performance difference cannot be attributed to differences in architecture or training procedure.

The coefficient α in Table 3 specifically applies to configurations with a distillation term, where it balances the cross-entropy against the distillation loss. The no-distillation configuration has no distillation term to balance against, so it uses $\alpha = 1.0$ (full cross-entropy) instead of 0.5. This follows the same full-weight principle applied to every other configuration in Table 3. Every "no-distillation" figure in the paper from this point onward uses $\alpha = 1.0$.

The sensitivity of the no-distillation configuration to α is presented in Section 3.1, and the sensitivity to β in Section 3.2.

2.9. Full training and evaluation pipeline

Algorithm 2 gathers the steps described from Section 2.5 to Section 2.8 into a single procedure, corresponding to Figure 1. With it, the reader can reproduce the order of teacher training, fold construction, seed use, and validation-based model selection without assembling information from several sections.

Algorithm 2. Full training and evaluation pipeline.
Input: preprocessed data of the 103 subjects, configuration set $\mathcal{C}$, seed set $\{1, \ldots, 5\}$.
Output: per-seed values $\bar{a}_{c,s}$ and the test result of each comparison.

1: Divide the 103 subjects, in code order, into 5 consecutive blocks $B_0, \ldots, B_4$
2: **for** $k = 0, \ldots, 4$: test $\leftarrow B_k$, validation $\leftarrow B_{(k+1) \bmod 5}$, training $\leftarrow$ the remaining three blocks
3: **for** $k = 0, \ldots, 4$: Riemannian alignment per subject within each partition of fold k
4: **for** $k = 0, \ldots, 4$: train teacher T_k on the training partition (120 epochs, fixed seed), keep the checkpoint with the best validation accuracy, then freeze T_k
5: **for** each seed s, each fold k, each configuration $c \in \mathcal{C}$:
6: set seed s for weight initialization, batch order, and sampling
7: class-balance the training trials of each subject by undersampling
8: **for** epoch $= 1, \ldots, 30$: update the student via (6) with the coefficients of c (QRD branch as in Section 2.3, V5 via Algorithm 1), T_k runs forward passes only
9: measure accuracy on the validation and test partitions
10: keep the per-subject test results at the epoch with the best validation accuracy
11: **for** each c and s: $\bar{a}_{c,s} \leftarrow$ mean over the 103 subjects (each subject is in the test set of exactly one fold)
12: **for** each pre-specified comparison (c_1, c_2): paired t-test on the 5 pairs $(\bar{a}_{c_1,s}, \bar{a}_{c_2,s})$, 95% CI, and d_z

Two properties of Algorithm 2 determine how the results should be read. The fold split in lines 1–2 and the teacher in line 4 do not depend on the seed. Every configuration and every seed thus share exactly the same five splits and five teachers. The difference between two configurations within the same seed therefore comes only from the knowledge-transfer mechanism and from training noise. The test set takes part in model selection at no step. Both the teacher and the student are selected on the validation partition, which is fully separated by subject from the test partition. The aggregation in line 11 reduces the 515 per-subject results to five values, matching the unit of inference stated in

Section 2.5.

3. Results and Discussion

The tests below use the seed as the unit of inference, as described in Section 2.5. Section 3.1 presents the effect of distillation and the degree of separation among mechanisms. Section 3.2 isolates each design axis together with the controls. Section 3.3 quantifies seed noise and ranking stability. Section 3.4 evaluates the configurations from a control-oriented perspective. Section 3.5 discusses the results. Table 4 summarizes the main results on PhysionetMI, with the difference from KDBase, the 95% CI, d_z, and ITR.

3.1. Distillation effect and the degree of separation among mechanisms

Table 4 gathers every distillation configuration on PhysionetMI into four blocks. The first block contains the baselines. The remaining three isolate, in turn, the dimensionality reduction, the relation strategy, and the kernel function of the QRD framework. The largest and clearest effect lies in whether distillation is present. The independently trained student reaches 74.93%, while every distilled configuration reaches about 81%. The improvement is 6.56 percentage points (95% confidence interval from 4.08 to 9.04 points, $p = 0.0018$, $d_z = 3.28$), far exceeding every other source of variation in the experiment. This improvement does not depend on choosing $\alpha = 1.0$ for the no-distillation configuration. Its accuracy rises monotonically with α and peaks at $\alpha = 1.0$. Any other choice of α only widens the gap (Appendix D). Distillation narrows most, but not all, of the gap to the teacher (84.98%). With KDBase, the remaining gap is 3.49 percentage points compared with 10.05 points for the non-distilled student, a narrowing of about 65% of the initial gap.

Table 4. Comparison of distillation mechanisms on PhysionetMI (5 seeds). Δ, the 95% confidence interval, and d_z are relative to KDBase. p is the paired t-test over seeds. Offline ITR is computed with $T = 3.0$ s.

Configuration	Accuracy (%)	Δ [95% CI]	p	d_z	ITR (bit/min)
Baselines					
No distillation	74.93 ± 2.30	$-6.56\ [-9.04; -4.08]$	0.002	-3.28	3.75
KDBase	81.49 ± 0.59	–	–	–	6.18
FeatureKD	80.05 ± 1.02	$-1.44\ [-2.43; -0.44]$	0.016	-1.79	5.58
RelationalKD	81.61 ± 0.84	$+0.12\ [-0.85; +1.08]$	0.754	$+0.15$	6.23
Quantum kernel, varying the dimensionality reduction					
V1 Dense (deterministic)	81.90 ± 0.71	$+0.41\ [-0.25; +1.07]$	0.158	$+0.78$	6.35
V2 Dense (learned projection)	81.49 ± 0.78	$+0.00\ [-0.54; +0.54]$	1.000	$+0.00$	6.18
Quantum kernel, varying the relation strategy					
V3 Anchor	81.25 ± 1.00	$-0.24\ [-0.88; +0.39]$	0.351	-0.47	6.07
V4 Anchor + kNN	81.42 ± 0.75	$-0.07\ [-0.70; +0.56]$	0.775	-0.14	6.15
V5 Guided (quantum kernel)	81.12 ± 0.71	$-0.37\ [-1.09; +0.36]$	0.231	-0.63	6.02
Fixed V5 strategy, varying the kernel function					
V5, cosine kernel	81.63 ± 0.63	$+0.14\ [-0.55; +0.83]$	0.609	$+0.25$	6.24
V5, RBF kernel	81.61 ± 0.58	$+0.13\ [-0.36; +0.61]$	0.511	$+0.32$	6.23

FeatureKD reaches 80.05%, 1.44 points below KDBase ($p = 0.016$, $d_z = -1.79$). The constraint of directly aligning features by squared error appears too rigid for EEG, forcing the student to reproduce representational components that are irrelevant to the classification task. The protocol detected this effect. Even so, a significant comparison does not guarantee equivalent power for other comparisons. With the observed standard deviation of the paired difference here (0.80 percentage points, $n = 5$), the minimum detectable effect at 80% power is about 1.35 percentage points. This is a sensitivity analysis performed after the data were collected, not an a priori calculation. The MDE of each comparison is given in Appendix E. The observed 1.44 points lies only slightly above this threshold.

The relation-based mechanisms do not separate from KDBase. Classical RelationalKD is 0.12 points above KDBase ($p = 0.754$). The five QRD variants with the quantum kernel range from 81.12% (V5 Guided) to 81.90% (V1 Dense), with the smallest p being 0.158. Every confidence interval for the relation-based mechanisms covers zero and spans about 1 to 2 percentage points. Distillation in general provides 6.56 percentage points, whereas all five QRD variants together with RelationalKD span only about 0.78 points around the KDBase mark. Section 3.2 analyzes each design axis behind this variation.

Replication on a second independent dataset (Cho2017) This part is an exploratory survey without a hypothesis or equivalence margin specified in advance. It is used only for qualitative comparison and does not replace the main comparison on PhysionetMI. The Cho2017 dataset [100], accessed via MOABB, consists of 52 subjects performing imagined left-hand and right-hand grasping. Three subjects (32, 46, 49) were excluded according to the default list used for the teacher [40], leaving 49 subjects. The protocol, architectures, and hyperparameters remain as in Section 2.8, and only the fold split changes with the number of subjects. The teacher reaches $75.07\% \pm 4.16\%$, lower and more variable than on PhysionetMI. Full accuracies for all thirteen configurations are given in Appendix F. Table 5 places Δ relative to KDBase side by side for the two datasets. The last column marks Yes if the sign direction agrees, regardless of statistical significance.

The distillation effect replicates on Cho2017. No KD reaches 70.32% versus 72.54% for KDBase, a difference of 2.22 percentage points (95% confidence interval $[1.39, 3.06]$, $p = 0.0018$). The direction and significance remain the same, but the magnitude is less than half of that on PhysionetMI. Part of the difference may come from the smaller teacher–student capacity gap. The teacher exceeds the non-distilled student by 4.75 points on Cho2017 versus 10.05 points on PhysionetMI, leaving less room for distillation. FeatureKD and RelationalKD also replicate almost exactly. FeatureKD is 1.38 points below KDBase ($p = 0.003$) versus 1.44 points on PhysionetMI, and RelationalKD does not differ from KDBase ($+0.03$ points, $p = 0.864$).

The difference among mechanisms remains smaller than seed noise, although the safety margin is thinner. The standard deviation across the twelve mechanisms (excluding No KD) is 0.37 percentage points, and the average standard deviation from seeds is 0.48 points. The ratio between the two is 0.77, compared with 0.61 on PhysionetMI. Six of the twelve rows in Table 5 change sign direction between the two datasets. In all six cases, Δ on at least one dataset is smaller than 0.5 percentage points, a region where the sign can easily flip from seed noise alone. The main kernel comparison also changes

Table 5. Side-by-side comparison of Δ relative to KDBase between PhysionetMI and Cho2017 for each configuration.

Configuration	PhysionetMI	Cho2017	Same direction
No KD	-6.56 (CI95% $[4.08; 9.04]$)	-2.22 (CI95% $[1.39; 3.06]$)	Yes
FeatureKD	-1.44 ($p = 0.016$)	-1.38 ($p = 0.003$)	Yes
RelationalKD	$+0.12$ ($p = 0.754$)	$+0.03$ ($p = 0.864$)	Yes
V1 Dense	$+0.41$ ($p = 0.158$)	-0.24 ($p = 0.405$)	No
V2 Dense	$+0.00$ ($p = 1.000$)	-0.18 ($p = 0.343$)	No
V3 Anchor	-0.24 ($p = 0.351$)	-0.51 ($p = 0.110$)	Yes
V4 Anchor + kNN	-0.07 ($p = 0.775$)	-0.26 ($p = 0.182$)	Yes
V5 Guided	-0.37 ($p = 0.231$)	-0.13 ($p = 0.626$)	Yes
Cosine kernel	$+0.14$ ($p = 0.609$)	-0.41 ($p = 0.030$)	No
RBF kernel	$+0.13$ ($p = 0.511$)	-0.18 ($p = 0.452$)	No
Direct kernel comparison within the same V5 strategy (not versus KDBase)			
Quantum versus cosine	-0.51 ($p = 0.096$)	$+0.28$ ($p = 0.389$)	No
Quantum versus RBF	-0.49 ($p = 0.060$)	$+0.05$ ($p = 0.903$)	No

direction. The quantum kernel is 0.51 points below cosine on PhysionetMI but 0.28 points above it on Cho2017 (95% confidence interval $[-0.52, 1.08]$, $p = 0.389$). Both confidence intervals cover zero. Two results with $p < 0.05$ on Cho2017 have no corresponding comparison on PhysionetMI and are only reported. The cosine kernel is 0.41 points below KDBase ($p = 0.030$), and V5 Guided with $\beta = 0.01$ is 0.37 points below KDBase ($p = 0.018$).

3.2. In-depth analysis of variant behavior

This part isolates each design axis of Table 4 and then checks the sensitivity of the conclusion to the coefficient β and summarizes the other design controls.

Relation-formation strategy and kernel function type In the dimensionality-reduction block, V1 uses a deterministic, parameter-free projection on both sides. V2 uses a frozen random W_T on the teacher side and a trained W_S on the student side (580 parameters, Table 2). This comparison is read as between two overall designs (Section 2.4). V1 is 0.41 percentage points above V2 in the direct comparison ($p = 0.085$), running counter to the design expectation but not reaching significance. The learned projection shows no measurable benefit over the deterministic design.

In the relation-strategy block, the three variants range from 81.12% to 81.42%, and none separates from KDBase (smallest p 0.231). The V5 strategy, expected to select the most informative relations, gives the lowest result in the group. V5 differs from V2–V4 in both the selection criterion and the candidate pool (64 versus 16 samples). The control variant *random_pool64* keeps the 64-sample pool but selects randomly, reaching 81.36%, not significantly different from either V2 or V5. The available data therefore do not rule out either factor.

In the kernel block, the V5 strategy is fixed to isolate the effect of the kernel alone. The three kernels span 0.51 percentage points, with cosine at 81.63%, RBF at 81.61%, and quantum at 81.12%. The paired t-test over five seeds, following the protocol fixed in advance, gives $p = 0.096$ (cosine minus quantum) and $p = 0.060$ (RBF minus quantum). Subjects within the same fold and seed share one model, and the five seeds are evaluated on the same subjects. The comparison was therefore also fitted with a mixed-effects regression with crossed random effects for subject and seed, and a subject-clustered bootstrap (10,000 iterations) gave the same range of results. Cosine minus quantum is 0.50 points (95% confidence interval $[0.00, 1.01]$, $p \approx 0.05$), and RBF minus quantum is 0.49 points ($[-0.01, 0.99]$, $p \approx 0.05$–0.06). The result of the pre-specified test is not superseded by these additional analyses. The two comparisons belong to one family, so a minimal Bonferroni correction brings the threshold to 0.025, which no p value reaches. The observed effect size nearly coincides with the ± 0.5-point equivalence margin. The correct conclusion under the rules fixed in advance is *not enough data to conclude in either direction*, neither equivalence nor a significant difference. Cosine and RBF do not differ ($p = 0.920$).

The coefficient β and the effective supervisory magnitude The coefficient β is checked in three steps. V5 Guided at $\beta = 0.01$ and $\beta = 0.05$ reaches 81.47% and 81.42%, neither separating from KDBase ($p = 0.91$ and $p = 0.82$) nor differing from the nominal level ($p = 0.15$ and $p = 0.40$). A gradient diagnostic on one fold and the first five epochs shows that the same $\beta = 0.02$ produces very different student-gradient norms (Appendix G). The norm is about 20 times larger for RBF than for cosine and about 200 times larger for quantum than for cosine. The three kernels therefore do not receive equivalent supervision when sharing β. To test the effect of this disparity directly, an additional experiment with a statistical plan fixed before running scaled β to 4.0 for cosine and 0.2 for RBF. These values bring their initial gradient magnitude level with that of the quantum kernel at $\beta = 0.02$. The experiment also swept $\beta \in \{0.1, 0.5\}$ for all three kernels. The pre-specified main comparison is quantum ($\beta = 0.02$) versus cosine ($\beta = 4.0$). The difference is -0.16 percentage points (95% confidence interval $[-0.51, +0.20]$, $p = 0.279$, $d_z = -0.56$). The lower bound exceeds the -0.5-point margin by about 0.01 points, so under the rules fixed in advance the conclusion remains

not enough data. Quantum versus RBF at $\beta = 0.2$ differs by $+0.13$ points ($p = 0.56$). All twelve configurations of this experiment lie between 81.32% and 81.87%, and no kernel shows a trend with β (Appendix G). Because the earlier numbers could not be reproduced exactly (Section 3.3), every comparison in this experiment uses only configurations rerun in the same batch. Rebalancing the supervisory magnitude does not change the conclusion about the kernel.

Interaction between kernel and relation strategy The comparisons above vary one axis at a time. To test whether the effect of the kernel depends on the relation strategy, an additional experiment crossed the three kernels with strategies V2–V5 (12 cells, $\beta = 0.02$, seeds 1–5). Its analysis plan was written before running, and the full results are in Appendix C. The pre-specified test is the kernel $\times$ strategy interaction in a two-way repeated-measures analysis of variance over seeds. The interaction is not significant ($F(6, 24) = 0.87$, $p = 0.53$). This result means that no interaction was detected at the power of a five-seed design. It does not show that the two axes are independent. Descriptively, the quantum kernel is below both classical kernels under all four strategies, by 0.06 to 0.60 points. However, none of these eight differences is significant (smallest $p = 0.065$), and the main effect of the kernel does not reach significance either ($F(2, 8) = 4.12$, $p = 0.059$).

Other design controls Three other design controls are reported in full in Appendix A. When cosine and RBF are recomputed on the projected space used by the quantum branch, the gap to the quantum kernel narrows to 0.13–0.24 points instead of widening. Fixing or splitting the shared circuit parameters θ does not change accuracy significantly ($p \geq 0.14$). Fixing W_T across seeds lowers the seed standard deviation of V2 from 0.78 to 0.51 points without significantly changing the mean. The same appendix includes a t-SNE analysis of the student features, whose result is in the same direction as the accuracy gap between students with and without distillation.

3.3. Seed noise and the stability of the method ranking

No relation variant or kernel function separated from KDBase in the part above. Is this because the true difference among mechanisms is small, or because seed noise exceeds the detection capability of the protocol? The standard deviation of the mean values across the twelve distillation mechanisms is 0.459 percentage points. The average standard deviation from changing the seed within the same mechanism is 0.750 points. The ratio between the two is 0.61. FeatureKD alone, the only mechanism with a significant effect, contributes 75% of the total sum of squared deviations among the twelve means. Excluding FeatureKD, the standard deviation across the remaining eleven mechanisms is 0.205 points, against a seed standard deviation of 0.726 points for the same group. The ratio then drops to 0.28. The gap among mechanisms is smaller than the noise that the training procedure itself generates.

A direct consequence is the question of whether the ranking drawn from a single run reflects the true order. Table 6 compares the ranking from a single run with the ranking from five seeds. The left column corresponds to the common reporting protocol, in which each configuration is run once across multiple folds with the fixed seed 20200220. This is the default value in the code, fixed before the five-seed protocol was built. The right column is taken from 5 independent seeds.

The two rankings are nearly uncorrelated. The Spearman coefficient is $\rho = 0.14$ with $p = 0.66$. The ranking from a single run is almost unable to predict the ranking from five seeds. V1 Dense stands at rank 11 of 12 in the single run but rises to first place over five seeds. V5 Guided goes in the opposite direction, from second to eleventh. The only configuration that keeps roughly the same position is FeatureKD at the bottom, also the only configuration with an effect exceeding the noise floor.

Table 6. Effect of the number of random repetitions on the method ranking (rank 6.5 = two tied configurations).

	One seed		Five seeds	
Configuration	Acc (%)	Rank	Acc (%)	Rank
V2 Dense	82.37	**1**	81.49	5.5
V5 Guided	81.86	2	81.12	**11**
RBF kernel	81.55	3	81.61	3.5
Cosine kernel	81.47	4	81.63	2
V5 Guided ($\beta = 0.01$)	81.40	5	81.47	7
V4 Anchor+kNN	81.32	6.5	81.42	8.5
RelationalKD	81.32	6.5	81.61	3.5
V5 Guided ($\beta = 0.05$)	81.21	8	81.42	8.5
V3 Anchor	81.14	9	81.25	10
KDBase	81.12	10	81.49	5.5
V1 Dense	81.06	11	81.90	**1**
FeatureKD	80.71	12	80.05	**12**

Five seeds are not necessarily enough for the ranking to converge either. Table 7 checks this with the cumulative ranking over the first k of seeds 1–5. Unlike the "One seed" column in Table 6, $k = 1$ here uses the first of the five main seeds, another independent data point illustrating the same phenomenon.

Table 7. Spearman rank correlation between the cumulative ranking over the first k seeds and the final ranking ($k = 5$).

k	ρ vs. $k = 5$	p	ρ vs. $k - 1$
1	0.615	0.033	—
2	0.692	0.013	0.930
3	0.853	< 0.001	0.895
4	0.916	< 0.001	0.958
5	1.000	< 0.001	0.916

The correlation with the final ranking rises monotonically from $\rho = 0.615$ at $k = 1$ to $\rho = 0.916$ at $k = 4$. Even so, adding exactly one seed from $k = 4$ to $k = 5$ still gives only $\rho = 0.916$, not 1.0. Five additional seeds (6–10) were run for five mechanisms (KDBase, RelationalKD, cosine V5, V1 Dense, and quantum V5). The correlation between the rankings at $k = 5$ and $k = 10$ is $\rho = 0.70$ ($p = 0.19$). The three top mechanisms at $k = 5$ (V1, cosine, RelationalKD) swap order among themselves at $k = 10$, with RelationalKD rising to first place. KDBase and the quantum kernel keep the fourth and fifth positions throughout from $k = 1$ to $k = 10$. Ranking instability therefore does not stop at five seeds for mechanisms with close results.

The sources of variation are not limited to the seed. Four configurations were rerun for the β experiment with the same training code, data, five seeds, and type of compute node. The results deviated from the original numbers by up to 0.33 percentage points (Table 8). A rerun of V2 within the factorial experiment deviates by -0.37 points (Appendix C). The two technical differences between the batches are the non-deterministic `cudnn.benchmark` mode and the number of CPU threads. The rerun limited the threads to 8, whereas the original run used the default of all cores. The quantum-minus-cosine gap at the same $\beta = 0.02$ is -0.51 points in the original batch but only -0.06 points ($p = 0.85$) in the rerun. The deviation between two runs of the same configuration is of the same order as the difference among mechanisms. The main tables keep the original numbers because they

form the analysis fixed in advance.

Table 8. Deviation between two batches of the same configurations, with the same five seeds.

Configuration	Original (%)	Rerun (%)	Difference (points)
KDBase	81.49 ± 0.59	81.71 ± 0.30	$+0.22$
V5, quantum kernel ($\beta = 0.02$)	81.12 ± 0.71	81.45 ± 0.56	$+0.33$
V5, cosine kernel ($\beta = 0.02$)	81.63 ± 0.63	81.51 ± 0.63	-0.12
V5, RBF kernel ($\beta = 0.02$)	81.61 ± 0.58	81.53 ± 0.57	-0.09

The no-distillation configuration is also sensitive to the seed. At $\alpha = 1.0$, its standard deviation across seeds is 2.30 percentage points, about four times that of the distilled configurations. This unusually large variance is what led to the α-sensitivity survey (Appendix D).

3.4. Evaluating the variants from a control-oriented perspective

Table 9 places training cost alongside the change in accuracy, measured on the same NVIDIA H100 GPU. The "$\approx$" values are the average of a single run, without a standard deviation across repeated measurements. The classical mechanisms, including the cosine and RBF kernels, need about 1.1 seconds per epoch. The variants using the quantum kernel need about 13.6 seconds, about 13 times more, because the data re-uploading structure in (8)–(9) requires more circuit-simulation passes per sample. This cost arises only during training, since the deployed model remains the classical student. Cost grows superlinearly with the number of qubits. Increasing the number of qubits to 8 or 12 does not improve the accuracy of V2 Dense (Appendix H).

Table 9. Training cost of the distillation mechanisms, measured by the average time per epoch.

Mechanism	Time/epoch (s)	Multiple	Δ Acc
KDBase	1.05	$1.0\times$	–
RelationalKD	1.10	$1.0\times$	$+0.12$ ($p = 0.75$)
Cosine kernel	1.10	$1.0\times$	$+0.14$ ($p = 0.61$)
RBF kernel	1.09	$1.0\times$	$+0.13$ ($p = 0.51$)
V5 Guided	$\approx$13.6	$\approx$**13**$\times$	-0.37 ($p = 0.23$)
V2 Dense	$\approx$13.6	$\approx$**13**$\times$	$+0.00$ ($p = 1.00$)

Following the criteria in Section 2.6, the ITR column of Table 4 converts accuracy into control-channel bandwidth. The κ coefficient is not listed, for the reason given in that section. ITR is a monotonic transformation of accuracy at a fixed window length, so it only restates the accuracy increase rather than providing additional empirical evidence. This is the offline ITR, computed with T_{window}. It does not yet add T_{infer} (under 1% of T_{window}, Table 10) or T_{command} (not measured, because the study is not tied to a specific actuator). The online ITR of a complete closed loop is therefore lower. With $T = 3.0$ s, the non-distilled student reaches 3.75 bit/min, while every distilled configuration lies between 5.58 and 6.35 bit/min, an increase in bandwidth of about 1.6 times. The gap among distillation mechanisms is 0.77 bit/min, coming largely from FeatureKD alone. Excluding this configuration, the gap narrows to 0.33 bit/min, about 5.4% of the bandwidth.

Table 10 decomposes the time budget of the closed loop. Inference time is measured with batch size one, taking the median and 95th percentile over 200 runs after 20 warm-up runs. The hardware and software configuration is given in Appendix I. On the HPC server CPU, the student needs 3.17 ms and the teacher 22.38 ms, that is, 0.11% and 0.75% of the 3000 ms window. The real-time constraint by itself does not rule out the teacher on a workstation. The difference between the two models lies in size, with the teacher 6.3 times larger in parameter count. On the hardware built into the device,

memory and energy are the tightening constraints. For the student, the analytically estimated peak activation is about 600 KB at float32, larger than the RAM of many low-end microcontroller families. The current architecture therefore has deployment potential but has not been confirmed on every class of embedded hardware.

Table 10. Decomposition of the closed control loop's time and resource budget via (24), measured on the HPC server CPU and on a Raspberry Pi 4.

Component	**Student (EEGNet)**	**Teacher (Ensemble-MI)**
Number of parameters	2,402	15.1 K
T_{window}	3000 ms	3000 ms
HPC server CPU		
T_{infer} (median)	3.17 ms	22.38 ms
T_{infer} (p95)	3.31 ms	22.63 ms
Ratio of T_{infer} to T_{window}	0.11%	0.75%
MACs per inference (batch=1)	$\approx$ 10.2 million	not computed
Peak activation (fp32, batch=1)	$\approx$ 600 KB	not computed
Precision	float32	float32
Raspberry Pi 4, ARM Cortex-A72 (mean $\pm$ standard deviation over 5 independent runs)		
T_{infer} (median)	12.23 ± 0.16 ms	85.30 ± 1.42 ms
T_{infer} (p95)	13.12 ± 0.43 ms	87.26 ± 1.48 ms
Ratio of T_{infer} to T_{window}	0.41%	2.84%
RAM after loading the model (RSS)	215.36 ± 0.50 MiB	219.11 ± 0.05 MiB
Peak RAM during inference (RSS)	241.80 ± 0.60 MiB	251.63 ± 0.11 MiB

On a Raspberry Pi 4 (ARM Cortex-A72, no GPU), the same student and teacher architectures are exported with TorchScript and measured with the same protocol, repeated over 5 independent runs. The exported student module is the exact training-time class, with a parameter count matching 2,402. The student's median T_{infer} is 12.23 ± 0.16 ms (95th percentile 13.12 ± 0.43 ms), occupying only about 0.41% of the signal window. The hardware was confirmed to be free of thermal throttling throughout the measurement. The teacher needs 85.30 ± 1.42 ms, about 7.0 times the student, nearly the same ratio as the 7.1 times measured on the server CPU. The teacher's inference time is therefore not the main barrier on this device. The measured process RAM of the two models is close (241.80 and 251.63 MiB) because most of it comes from the Python and PyTorch overhead (Appendix I). The Raspberry Pi 4 does not belong to the ultra-low-power microcontroller class as in [17]. This measurement therefore does not yet confirm feasibility on the lowest tier of hardware, but it is direct measurement evidence on real embedded hardware.

3.5. Discussion

(i) Main findings. Under the 30-epoch training budget, knowledge distillation provides a large and replicable benefit, 6.56 percentage points on PhysionetMI and 2.22 points on Cho2017. Once distillation is present, the differences among the twelve specific mechanisms lie within the noise of the training process itself. The ranking from a single run is almost uncorrelated with the ranking over five seeds ($\rho = 0.14$). Two batches of the same configuration also deviate by up to 0.37 points. The quantum kernel, the strictest test case, does not separate from the classical kernels, even after the supervisory magnitude is rebalanced. Under the rules fixed in advance, this comparison remains inconclusive rather than showing equivalence. The factorial experiment also detected no kernel $\times$ strategy interaction, within the power of a five-seed design.

(ii) Comparison with other studies. The observed seed noise is consistent with warnings from

computer vision and machine learning in general [48, 49] and from deep learning on neuroimaging data [50]. This study extends those warnings to the comparison of distillation mechanisms for motor imagery EEG, with concrete measurements of ranking stability. A real-world evaluation of ten deep motor imagery decoders under a soft real-time protocol likewise found that rankings shifted between offline and online settings [101]. In that study, inter-subject variability also increased markedly in the online setting. Together with the seed-induced ranking instability measured here, this suggests that offline single-run rankings should be treated as provisional. Earlier relational distillation work for EEG proposed new mechanisms within its own setting [66, 67]. The results here do not directly contradict that work, because the datasets, teachers, and protocols differ. Even so, differences of a few tenths of a percentage point among mechanisms lie within the noise floor measured here. They are hard to detect reliably from a single run. For the quantum kernel, the absence of an advantage is consistent with the observation that classical models learning from data can match quantum models [72]. Quantum relational distillation was proposed and evaluated on vision and language [89]. On EEG with the protocol used here, it does not separate from the classical kernels. Embedded deployment studies compare CPU, GPU, and FPGA platforms by inference time, power, and memory [18] or demonstrate real-time EEG decoding on edge devices [19]. The Pi 4 measurement here is narrower, covering only the forward pass of the student. The absolute accuracies in this paper are measured under a subject-level transductive protocol (Section 2.5), so they are not directly comparable with figures obtained under online or inductive protocols.

The directly comparable numerical reference is the teacher study itself [40]. That study uses the same 103 PhysionetMI subjects, the same two-class task, the same preprocessing code base, and the same subject-wise 5-fold split into training, validation, and test blocks. It reports 86.36% for the ensemble with seven first-stage models and 82.09% for a single EEGNet trained for 120 epochs with a learning-rate schedule. The teacher reproduced in this paper reaches 84.98%, 1.38 percentage points lower. The distilled students in Table 4 (81.12% to 81.90%) remain below that single EEGNet. The no-distillation student (74.93%) is about 7 points below it. The training budget explains at least part of this gap. The students here are trained for 30 epochs at a fixed learning rate of 0.005 on 3-s windows. The reference recipe uses 120 epochs, a decaying learning rate, and 4-s windows. The teacher here is also trained on 3-s windows. The 6.56-point benefit of distillation is therefore measured relative to a student trained under a short budget and should not be read as the gain over a fully tuned EEGNet. This does not affect the comparisons among mechanisms, because every mechanism shares the same budget. It does, however, limit the claims about absolute accuracy.

(iii) Interpretation and implications. Once logit distillation is present, additional information from relations among samples appears to contribute little to final accuracy, whether relations are measured in Euclidean, cosine, RBF, or Hilbert space. That this pattern also holds for the quantum kernel suggests that the limited role of relation structure is not specific to one representation space, at least among the four spaces examined. This does not mean the representation space plays no role at all, a stronger claim than the data allow. For system design, the choice of specific distillation mechanism changes control-channel bandwidth by only about 0.33 bit/min when FeatureKD is excluded, which is unlikely to be operationally meaningful. Training cost and implementation complexity are therefore more useful selection criteria. With the current implementation, the roughly 13-fold cost of the quantum branch is not accompanied by a measurable benefit. Methodologically, comparisons of distillation mechanisms for EEG need multiple seeds, confidence intervals, and checks of the deviation between reruns, rather than a single score.

(iv) Strengths and limitations. The strengths of the study include the subject as the statistical unit, a statistical plan fixed in advance for the kernel comparisons, and design controls. They also include an exploratory replication on a second dataset and a measurement on real embedded hardware. The first limitation is that per-subject covariance alignment makes the protocol offline and transductive at the subject level. The accuracy, ITR, and latency figures should be read under this protocol. Support-

ing an online deployment claim directly requires a protocol that uses only data available before each inference. With five seeds, the minimum detectable effect at 80% power among the distilled mechanisms lies between about 0.65 and 1.35 percentage points depending on the comparison (Appendix E). This is larger than most differences among mechanisms and larger than the ± 0.5-point equivalence margin. The student training budget (30 epochs, a fixed learning rate, 3-s windows) is shorter than the reference recipe, so the absolute accuracies are below the published single-EEGNet figure. The main comparison is carried out only on PhysionetMI with a two-class problem, and the survey on Cho2017 is exploratory. The quantum component takes only one circuit form with four qubits run on a simulator, whereas the performance of quantum kernels depends on the feature map [73]. The conclusion therefore does not extend to quantum machine learning in general. Non-deterministic training means that the numbers are not reproducible to the last digit. The β rebalancing ratio was measured early in training and may drift later. The controls on θ, W_T, and V5's candidate pool do not entirely rule out the corresponding theoretical risks. The study has also not been validated in a closed control loop with real users.

4. Conclusion

This paper presents a controlled evaluation protocol for knowledge distillation mechanisms on motor imagery EEG, using the QRD framework as the test bed. The QRD framework separates the relation-formation strategy from the representation space and keeps every relation kernel, classical or quantum, outside the deployed model. Five strategies and three kernel function types were compared under uniform conditions over five seeds, with the subject as the statistical unit.

The study supports three conclusions. Under the 30-epoch training budget, knowledge distillation provides a large benefit, which replicates in direction on the exploratory Cho2017 dataset. The differences among specific distillation mechanisms are smaller than the noise from seeds and from reruns, large enough to reverse the method ranking between runs. No quantum variant provides a statistically significant advantage over the classical kernels, even after rebalancing the supervisory magnitude. The factorial experiment detected no interaction between kernel and relation strategy either. The paper does not establish equivalence between quantum and classical kernels. The new contribution to the field is quantitative evidence that, in this controlled setting, rankings of distillation mechanisms for EEG drawn from a single run are unreliable. The study also contributes a control-oriented evaluation procedure and a latency measurement on embedded hardware. This result does not refute published single-run results. It only shows that differences of a few tenths of a percentage point need confirmation over multiple seeds. The main limitations, including the transductive protocol, five seeds, and a single main dataset, are stated in Section 3.5.

Future work should evaluate an online protocol that uses only data available before each inference. It should also validate the decoder in a closed control loop with real users, measuring end-to-end latency, memory, and energy. The main comparison should be repeated on a second dataset with a publicly registered statistical plan. Extending the study to the multi-class problem and to larger quantum circuits, when the infrastructure allows, is a further direction. On the methodological front, a protocol that determines the minimum number of repetitions from the measured noise floor, together with deterministic training, would make method comparisons in this field more reliable.

Reproducibility. The training, preprocessing, and statistical analysis code, the list of 103 subject-to-fold assignments, the seeds, the five teacher checkpoints, and the per-seed results are publicly available at https://github.com/nguyenphuonghuy/qrd_eeg as of the submission of this revision. The software environment is version-pinned (PyTorch 2.7.1, PennyLane, braindecode). Because GPU training is non-deterministic, reruns deviate within the range shown in Table 8 and Appendix C.

A. Additional design controls

This appendix gives the qualitative analysis of the feature space and the full results of the three design controls summarized in Section 3.2.

Qualitative analysis of the feature space (t-SNE) Figure 4 visualizes the student's intermediate features with t-SNE for four configurations (no distillation, KDBase, FeatureKD, and V5 Guided). All panels use the same fold 0 (900 samples, 20 subjects), the same seed 1, and the same t-SNE hyperparameters (perplexity $= 30$, random_state $= 42$). The accuracy and silhouette in each panel's title belong to this single run. The cluster structure of the three distilled configurations is more clearly separated than the independent student (silhouette 0.052 to 0.065 versus 0.013), in the same direction as the accuracy gap in Table 4. Among the three distilled configurations, the silhouette scores are close even though the cluster shapes differ. T-SNE does not preserve global distances, so differences in cluster shape are not regarded as evidence. The silhouette index on the original 144-dimensional space is the quantitative basis.

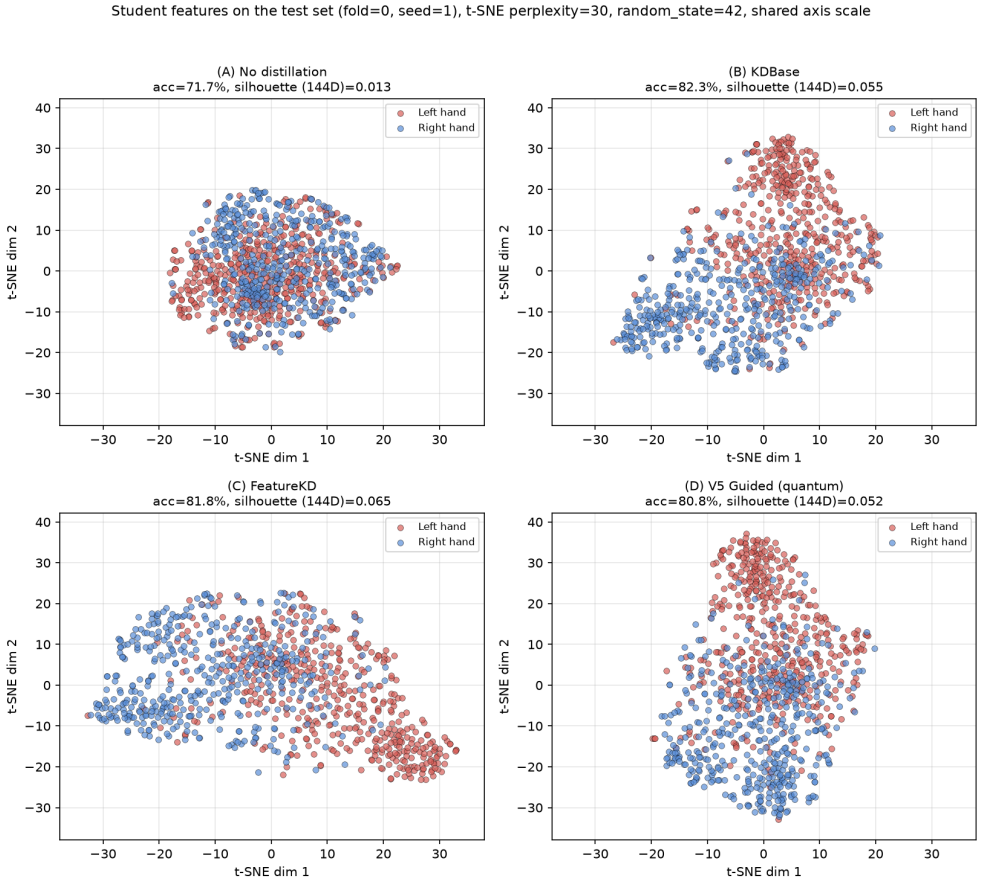


Fig. 4. Qualitative analysis of the feature space using t-SNE visualization on the student's 144-dimensional intermediate features, extracted on the test set at the epoch with the best validation accuracy.

Input space of the classical kernels The gap between the quantum kernel and the two classical kernels in Section 3.2 might stem from a non-equivalent input space. Cosine and RBF use the raw 144-dimensional features, whereas the quantum kernel uses the space projected through W_S. Cosine and RBF were recomputed on features projected through the learned W_S and the fixed W_T, identical to the quantum branch. The results are projected cosine $81.25 \pm 0.70\%$ and projected RBF $81.36 \pm 0.91\%$. Neither differs significantly from the original version (cosine $\Delta = -0.38$, $p = 0.168$; RBF $\Delta = -0.25$, $p = 0.293$) or from quantum V5 (projected cosine $+0.13$, $p = 0.647$; projected RBF $+0.24$,

$p = 0.312$). The gap between quantum and the classical kernels narrows to 0.13–0.24 points rather than widening.

Co-adaptation of the shared circuit parameters (θ) Sharing θ could let training flatten both kernels instead of forcing the student to reproduce the teacher's relation structure (Section 2.3). V2 Dense was trained further with θ entirely fixed (*shared_fixed*, $81.73 \pm 0.49\%$) and with a fixed θ_T separated from a trainable θ_S (*split*, $81.35 \pm 0.87\%$), each with 5 seeds $\times$ 5 folds. Compared with the original shared θ (*shared*, $81.49 \pm 0.78\%$), no pair differs significantly ($p = 0.143$, 0.615, and 0.215). This result does not prove that the degeneracy risk never occurs. It only shows no measurable effect on accuracy at the scale examined.

Initialization of W_T V2 Dense was retrained with W_T fixed at a single value while still sweeping all five seeds. The standard deviation across seeds drops from 0.78 to 0.51 percentage points, while the mean accuracy does not differ significantly (81.20% versus 81.49%, $p = 0.22$). A substantial but not complete part of the seed noise in the quantum branch comes from W_T changing with the global seed. This control does not yet measure the variation across multiple initializations of W_T at the same training seed.

B. Implementation details

Initialization of the projections. Both W_T and W_S are PyTorch `nn.Linear` layers initialized with the library's default distribution, without any further normalization step. The weights follow a Kaiming-uniform distribution and the bias a uniform distribution, both scaled by the input dimensionality. This initialization step occurs after several other calls that consume random numbers (data loading, teacher and student initialization) within the same global random number stream seeded per run. As a result, W_T is likely to differ across the experiment's five seeds, an unintended consequence of the global seeding scheme.

Theoretical limits of the projection and angle encoding. The random linear projection at a $144 \to 4$ compression ratio has no Johnson–Lindenstrauss-type theoretical guarantee of preserving relation structure. For a circuit with multiple qubits and multiple entangling layers, the injectivity of the overall mapping is also not rigorously proven merely from bounding the angle domain in (7).

Circuit degeneracy diagnostic. The difference of 0.474 between the quantum kernel and the corresponding cosine product is compared with a threshold of 0.05. This threshold is a small conventional marker, used to detect complete degeneration into a classical kernel (a difference near 0), and is not based on a formal statistical test.

Diagonal pairs in V1 and V2. The index set $\mathcal{I}_{\text{V1}}$ includes the diagonal pairs (i, i). These elements always have $R_T^{(Q)}(i,i) = R_S^{(Q)}(i,i) = 1$ by the definition of fidelity, so they contribute exactly 0 to $\mathcal{L}_{\text{QRD}}$, carry no information, and generate no gradient. They only slightly dilute the average in (12) by increasing $|\mathcal{I}|$.

Tied distances in V4. When `torch.topk` encounters tied distances among neighbors, the order returned depends on PyTorch's implementation (stable but with no statistically meaningful priority). With continuous features as used here, the probability of an exact distance tie is almost negligible on real data.

C. Factorial experiment: kernel $\times$ relation strategy

Table 11 reports all 12 cells of the factorial experiment summarized in Section 3.2. Every cell uses PhysionetMI, 5 folds, 30 epochs, seeds 1–5, and the nominal $\beta = 0.02$, with the other hyperparameters as in Table 3. The three V5 cells are reused from the β experiment (Appendix G), run with the same code and environment. As in the main table, cosine and RBF are computed on the raw 144-dimensional

features and the quantum kernel on the projected space. "Kernel" here therefore means the kernel together with its input space. The cosine cells combine three batches because the first job reached its time limit. The merging rule, one source per seed and fold, was fixed and committed before any result was examined.

Table 11. Factorial experiment: accuracy (%, mean ± standard deviation over five seeds) and paired differences of the quantum kernel from the classical kernels (percentage points, 95% CI).

	V2 Dense	**V3 Anchor**	**V4 Anchor + kNN**	**V5 Guided**
Quantum	81.12 ± 0.82	81.39 ± 0.66	81.31 ± 0.88	81.45 ± 0.56
Cosine	81.72 ± 0.89	81.62 ± 0.36	81.59 ± 0.66	81.51 ± 0.63
RBF	81.67 ± 0.48	81.72 ± 0.50	81.91 ± 0.57	81.53 ± 0.57
Quantum − cosine	-0.60 $[-1.65; +0.45]$	-0.23 $[-0.76; +0.30]$	-0.28 $[-0.73; +0.18]$	-0.06 $[-0.81; +0.70]$
Quantum − RBF	-0.55 $[-1.15; +0.06]$	-0.34 $[-0.91; +0.23]$	-0.60 $[-1.48; +0.29]$	-0.07 $[-0.59; +0.44]$

The interaction test gives $F(6, 24) = 0.87$, $p = 0.53$. The main effect of the strategy gives $F(3, 12) = 0.59$, $p = 0.63$, and the main effect of the kernel gives $F(2, 8) = 4.12$, $p = 0.059$. All eight differences in the lower block are negative, but every confidence interval contains 0. The minimum detectable effect at 80% power for these eight comparisons ranges from 0.61 to 1.42 points, so five seeds are not enough to resolve differences of the observed size. The V2 quantum cell rerun here is 0.37 points below its value in Table 4 (81.12% versus 81.49%). V3 and V4 deviate by $+0.14$ and -0.11 points. These deviations add to the rerun spread reported in Table 8.

D. Sensitivity of the no-distillation configuration to α

The no-distillation configuration was trained further at four other values of α. Table 12 shows that accuracy rises monotonically with α, from near-chance level at $\alpha = 0$ (no supervisory signal from the true labels) to 74.93% at $\alpha = 1.0$. If the distillation improvement were computed at $\alpha = 0.5$, the independent student would reach only 63.06% and the improvement would rise to 18.43 percentage points. This difference does not reflect two comparable estimates of the same effect. It reflects that $\alpha = 0.5$ does not give the no-distillation configuration enough weight on the one loss term it has. $\alpha = 1.0$ is consistent with the full-weight principle applied to every other configuration. The standard deviation across seeds also rises sharply as α decreases, reaching 5.03 points at $\alpha = 0.5$.

Table 12. Sensitivity of the no-distillation configuration to the coefficient α (PhysionetMI, 5 seeds × 5 folds).

α	**Accuracy (%)**
0.00	49.89 ± 0.34
0.25	51.73 ± 2.68
0.50	63.06 ± 5.03
0.75	71.45 ± 3.74
1.00 (used for the main baseline)	$\mathbf{74.93 \pm 2.30}$

E. Minimum detectable effects

Table 13 reports the minimum detectable effect (MDE) at 80% power for each comparison on PhysionetMI, with a two-sided paired t-test, $\alpha = 0.05$, and $n = 5$ seeds, computed with the non-central t distribution. The analysis uses the observed standard deviation of the paired difference, so it is a post hoc analysis rather than a sample-size calculation made before running. Among the distilled mechanisms, the MDE lies between 0.65 and 1.35 percentage points depending on the standard deviation of each pair. For the three kernel comparisons and V1 versus V2, the MDE ranges from 0.68 to 0.87 points, all larger than the ± 0.5-point equivalence margin. The five-seed design therefore

lacks the power to detect reliably a difference of the size of the equivalence margin, consistent with the not-enough-data conclusion in Section 3.2.

Table 13. Minimum detectable effect at 80% power (PhysionetMI, post hoc analysis). The observed Δ and the standard deviation of the paired difference are in percentage points.

Comparison	Observed Δ	SD of difference	MDE 80%
No KD vs. KDBase	-6.56	2.00	3.36
FeatureKD vs. KDBase	-1.44	0.80	1.35
RelationalKD vs. KDBase	$+0.12$	0.78	1.30
V1 Dense vs. KDBase	$+0.41$	0.53	0.89
V2 Dense vs. KDBase	$+0.00$	0.44	0.73
V3 Anchor vs. KDBase	-0.24	0.51	0.86
V4 Anchor + kNN vs. KDBase	-0.07	0.50	0.85
V5, quantum kernel vs. KDBase	-0.37	0.58	0.98
V5, cosine kernel vs. KDBase	$+0.14$	0.56	0.94
V5, RBF kernel vs. KDBase	$+0.13$	0.39	0.65
Quantum vs. cosine (V5)	-0.50	0.52	0.87
Quantum vs. RBF (V5)	-0.49	0.42	0.71
V1 vs. V2	$+0.41$	0.40	0.68

F. Full results on Cho2017

The seven configurations using the quantum kernel (V1–V5 and the two β variants of V5) were trained with the quantum circuit described in Section 2.3. The six classical configurations do not depend on the quantum circuit and keep their existing figures. Table 14 reports the accuracy of all thirteen configurations. Every distilled configuration lies between 71.17% and 72.57%. The three configurations with $p < 0.05$ relative to KDBase are FeatureKD, the cosine kernel, and V5 Guided at $\beta = 0.01$. The latter two have no corresponding comparison on PhysionetMI and are only reported.

Table 14. Results on Cho2017 (49 subjects, 5 independent seeds). Δ, the 95% confidence interval, and d_z are relative to KDBase.

Configuration	Accuracy (%)	Δ [95% CI]	p	d_z
No KD	70.32 ± 0.64	-2.22 $[-3.06; -1.39]$	0.0018	-3.30
KDBase	72.54 ± 0.21	—	—	—
FeatureKD	71.17 ± 0.69	-1.38 $[-1.98; -0.77]$	0.0032	-2.83
RelationalKD	72.57 ± 0.42	$+0.03$ $[-0.38; +0.43]$	0.864	$+0.08$
V1 Dense	72.31 ± 0.69	-0.24 $[-0.94; +0.47]$	0.405	-0.42
V2 Dense	72.37 ± 0.43	-0.18 $[-0.63; +0.28]$	0.343	-0.48
V3 Anchor	72.03 ± 0.63	-0.51 $[-1.20; +0.18]$	0.110	-0.92
V4 Anchor + kNN	72.28 ± 0.56	-0.26 $[-0.71; +0.19]$	0.182	-0.72
V5 Guided	72.41 ± 0.65	-0.13 $[-0.83; +0.57]$	0.626	-0.24
V5 Guided ($\beta = 0.01$)	72.17 ± 0.38	-0.37 $[-0.63; -0.11]$	0.018	-1.74
V5 Guided ($\beta = 0.05$)	72.36 ± 0.27	-0.19 $[-0.42; +0.05]$	0.091	-0.99
Cosine kernel	72.13 ± 0.30	-0.41 $[-0.76; -0.06]$	0.030	-1.47
RBF kernel	72.36 ± 0.52	-0.18 $[-0.79; +0.42]$	0.452	-0.37

G. The coefficient β and the effective supervisory magnitude

Gradient diagnostic. On 1 fold and the first 5 epochs of PhysionetMI, the diagnostic measures the raw value of $\mathcal{L}_{\text{QRD}}$ before multiplying by β and the L_2 norm of the gradient that $\beta\,\mathcal{L}_{\text{QRD}}$ alone induces on the student's parameters. A separate backpropagation pass is run on the first batch of each epoch. Cosine and RBF use the V5 Guided configuration. The quantum kernel uses V2 Dense, with the same `dense` strategy and differing from V5 only in the index set, which does not affect the measured quantity. Table 15 shows that the same $\beta = 0.02$ produces supervision differing by up to two orders

of magnitude between the quantum kernel and cosine, with RBF closer to the quantum scale. The diagnostic is not repeated over seeds, so the specific numbers are only indicative. The magnitude of the difference (tens to hundreds of times) is still clear enough to rule out equivalent supervision across the three kernels.

Table 15. Diagnostic of the effective supervisory magnitude of $\mathcal{L}_{\text{QRD}}$ by kernel function type, at the shared nominal $\beta = 0.02$.

Kernel	Raw $\mathcal{L}_{\text{QRD}}$	$\|\nabla_{\text{student}}\|_2$	Multiple (vs. cosine)
Cosine	2.16×10^{-3}	7.54×10^{-5}	$1\times$
RBF	3.13×10^{-2}	1.47×10^{-3}	$\approx 20\times$
Quantum	1.41×10^{-1}	1.51×10^{-2}	$\approx 200\times$

Gradient-matched experiment. The statistical plan of the experiment was fixed before running. The β values for cosine (4.0) and RBF (0.2) were derived from the ratios in Table 15 so that their initial gradient magnitude matches the quantum kernel at $\beta = 0.02$. Every configuration, including KDBase and the three configurations at $\beta = 0.02$, was rerun in the same batch with the same five seeds. Table 16 reports all twelve configurations. Every configuration lies between 81.32% and 81.87%. No kernel shows a trend with β. Relative to the rerun KDBase, 10 of 11 configurations have $p > 0.05$. Cosine at $\beta = 0.5$ alone has $p = 0.044$, one of 11 comparisons without multiple-comparison correction, while the expected number of false positives at $\alpha = 0.05$ is about 0.55. This result is not used as a claim. The main comparison of the experiment is presented in Section 3.2.

Table 16. The β experiment (PhysionetMI, V5 Guided, 5 seeds, one batch). Δ and the 95% confidence interval are relative to the rerun KDBase.

Kernel	β	Accuracy (%)	Δ [95% CI]	p
KDBase	–	81.71 ± 0.30	–	–
Quantum	0.02	81.45 ± 0.56	$-0.26\ [-0.65; +0.13]$	0.141
Quantum	0.1	81.32 ± 0.72	$-0.39\ [-1.13; +0.35]$	0.218
Quantum	0.5	81.42 ± 0.72	$-0.29\ [-0.82; +0.24]$	0.204
Cosine	0.02	81.51 ± 0.63	$-0.20\ [-0.82; +0.42]$	0.415
Cosine	0.1	81.51 ± 0.69	$-0.21\ [-0.84; +0.43]$	0.415
Cosine	0.5	81.33 ± 0.32	$-0.38\ [-0.74; -0.02]$	0.044
Cosine	4.0 (gradient-matched)	81.61 ± 0.63	$-0.10\ [-0.52; +0.32]$	0.544
RBF	0.02	81.53 ± 0.57	$-0.19\ [-0.65; +0.28]$	0.333
RBF	0.1	81.86 ± 0.41	$+0.14\ [-0.29; +0.57]$	0.410
RBF	0.2 (gradient-matched)	81.32 ± 0.66	$-0.39\ [-0.95; +0.16]$	0.120
RBF	0.5	81.87 ± 0.33	$+0.16\ [-0.10; +0.42]$	0.168

H. Training cost as a function of the number of qubits

A preliminary survey measures training time as a function of the number of qubits ($n_{\text{qubits}} \in \{4, 8, 12, 16\}$, 1 fold, 3 epochs, every other hyperparameter fixed) using V2 Dense. V2 and V5 have the same measured cost because they select the same number of sample pairs for the kernel, differing only in the selection criterion. The number of qubits affects only the cost of each kernel evaluation, so the trend measured on V2 can be extrapolated to the other variants for cost rather than for accuracy. Table 17 shows that cost increases superlinearly. The multiple between consecutive levels (about $2.8\times$, $1.8\times$, and $4.5\times$ for each 4-qubit step) is much lower than the $16\times$ that statevector simulation predicts for the circuit part alone. This suggests that fixed costs (data loading, the classical backbone, PennyLane overhead) still account for a substantial share at this scale. At $n_{\text{qubits}} = 16$, the measurement did

not complete 3 epochs within 20 minutes. A full run at this level is estimated at about 65 hours for one configuration, beyond the capacity of the current infrastructure. At 8 and 12 qubits, full accuracy was measured for V2 Dense. No pair among the three qubit levels differs significantly (including 8 versus 12, $p = 0.491$). With $n = 5$, this result only shows that no trend with the number of qubits has been detected.

Table 17. Training cost and accuracy as a function of the number of qubits (V2 Dense). Time/epoch measured in the preliminary survey (1 fold × 3 epochs). Accuracy measured in full (5 seeds × 5 folds × 30 epochs) for $n_{\text{qubits}} \leq 12$.

n_{qubits}	Time/epoch (s)	Multiple vs. previous level	Accuracy (%)	p (vs. $n = 4$)
4	≈ 13.6	–	81.49 ± 0.78	–
8	36–40	$\approx 2.8\times$	81.36 ± 0.38	0.691
12	67–71	$\approx 1.8\times$	81.16 ± 0.75	0.332
16	308–315	$\approx 4.5\times$	not measured (3 epochs not completed)	–

I. Latency measurement details

HPC server CPU. The measurement runs on a shared compute node of the HPC cluster with an AMD EPYC 9224 CPU (24 cores per socket, 2 sockets, max clock ≈ 3.7 GHz). It uses 8 threads, the default number of CPUs allocated to the process, with PyTorch 2.7.1, the default Intel oneAPI MKL BLAS backend, float32 precision, and Linux kernel 5.14 (Enterprise Linux 9). The configuration was confirmed on the same node, although a slight change in OS patches between measurements cannot be ruled out. The student's 3.17 ms is on the same order of magnitude as the 2.95 ms reported by another EEGNet deployment on an ultra-low-power microcontroller [17]. These are two different architectures, datasets, and hardware types, so this similarity does not prove equivalent deployment conditions. Inference time on a microcontroller also depends on the number of MACs, intermediate RAM, precision, the kernel library, and the specific microcontroller. The MACs and peak activation in Table 10 are computed analytically from the EEGNet architecture, not measured on hardware.

Raspberry Pi 4. The measurement uses the same protocol as on the server CPU (200 inferences after 20 warm-up runs, batch size one, no special thread-count configuration). It was repeated over 5 independent runs and re-verified with a sixth run, which gave a consistent result. The student's latency ranges from 11.7 to 15.0 ms across all measurements, at 42.3°C, without thermal throttling, with the CPU frequency held at the maximum 1800 MHz. The measurement uses randomly initialized weights, because inference time and memory depend only on the architecture and tensor sizes. The teacher's peak RAM (251.63 ± 0.11 MiB) is only about 4.1% higher than the student's even though the teacher has 6.3 times as many parameters. Most of the measured RAM comes from the fixed overhead of the Python interpreter and the PyTorch runtime. This is the RAM of the entire process, which differs fundamentally from the estimated ≈ 600 KB of pure student activation. The two figures should not be compared directly.

Author Contribution: All authors contributed equally to the main contribution of this paper. All authors read and approved the final paper.

Funding: This research received no external funding.

Acknowledgment: This work was supported in part by the Thai Nguyen University of Technology, and was conducted at the "Northern, Central and Southern Region Shared Key Laboratory", specializing in the fields of Internet of Things (IoT), High-Performance Computing (HPC) and Unmanned Aerial Vehicles (UAVs) Viet Nam.

Conflicts of Interest: The authors declare no conflict of interest.

References

[1] S. Chen, M. Chen, X. Wang, X. Liu, B. Liu, and D. Ming, "Brain–computer interfaces in 2023–2024," *Brain-X*, vol. 3, no. 1, p. e70024, Mar. 2025, https://doi.org/10.1002/brx2.70024.

[2] L. Tonin and J. D. R. Millán, "Noninvasive Brain–Machine Interfaces for Robotic Devices," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, no. 1, pp. 191–214, May 2021, https://doi.org/10.1146/annurev-control-012720-093904.

[3] Y. Ding and B. He, "Brain–computer interfaces for robotic control," *National Science Review*, p. nwag269, May 2026, https://doi.org/10.1093/nsr/nwag269.

[4] K. Värbu, N. Muhammad, and Y. Muhammad, "Past, Present, and Future of EEG-Based BCI Applications," *Sensors*, vol. 22, no. 9, p. 3331, Apr. 2022, https://doi.org/10.3390/s22093331.

[5] W. Pang, C. Yuan, Y. Zheng, T. Zhong, and P. Lai, "Applications over the horizon — Advancements and challenges in brain-computer interfaces," *The Innovation Life*, vol. 2, no. 1, p. 100058, 2024, https://doi.org/10.59717/j.xinn-life.2024.100058.

[6] A. Palumbo, V. Gramigna, B. Calabrese, and N. Ielpo, "Motor-Imagery EEG-Based BCIs in Wheelchair Movement and Control: A Systematic Literature Review," *Sensors*, vol. 21, no. 18, p. 6285, Sep. 2021, https://doi.org/10.3390/s21186285.

[7] Z. Lei, B. Y. Tan, N. P. Garg, L. Li, A. Sidarta, and W. T. Ang, "An Intention Prediction Based Shared Control System for Point-to-Point Navigation of a Robotic Wheelchair," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 8893–8900, Oct. 2022, https://doi.org/10.1109/LRA.2022.3189151.

[8] X. Mai, J. Ai, M. Ji, X. Zhu, and J. Meng, "A hybrid BCI combining SSVEP and EOG and its application for continuous wheelchair control," *Biomedical Signal Processing and Control*, vol. 88, p. 105530, Feb. 2024, https://doi.org/10.1016/j.bspc.2023.105530.

[9] A. Casey, H. Azhar, M. Grzes, and M. Sakel, "BCI controlled robotic arm as assistance to the rehabilitation of neurologically disabled patients," *Disability and Rehabilitation: Assistive Technology*, vol. 16, no. 5, pp. 525–537, Jul. 2021, https://doi.org/10.1080/17483107.2019.1683239.

[10] P. D. E. Baniqued, E. C. Stanyer, M. Awais, A. Alazmani, A. E. Jackson, M. A. Mon-Williams, F. Mushtaq, and R. J. Holt, "Brain–computer interface robotics for hand rehabilitation after stroke: A systematic review," *Journal of NeuroEngineering and Rehabilitation*, vol. 18, no. 1, p. 15, Jan. 2021, https://doi.org/10.1186/s12984-021-00820-8.

[11] Y. Lu, L. Bi, and H. Li, "Model Predictive-Based Shared Control for Brain-Controlled Driving," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 2, pp. 630–640, Feb. 2020, https://doi.org/10.1109/TITS.2019.2897356.

[12] Y. An, J. Wong, and S. H. Ling, "Development of real-time brain-computer interface control system for robot," *Applied Soft Computing*, vol. 159, p. 111648, Jul. 2024, https://doi.org/10.1016/j.asoc.2024.111648.

[13] L. Tonin, F. C. Bauer, and J. Del R. Millan, "The Role of the Control Framework for Continuous Teleoperation of a Brain–Machine Interface-Driven Mobile Robot," *IEEE Transactions on Robotics*, vol. 36, no. 1, pp. 78–91, Feb. 2020, https://doi.org/10.1109/TRO.2019.2943072.

[14] Y. Sun, X. Chen, B. Liu, L. Liang, Y. Wang, S. Gao, and X. Gao, "Signal acquisition of brain–computer interfaces: A medical-engineering crossover perspective review," *Fundamental Research*, vol. 5, no. 1, pp. 3–16, Jan. 2025, https://doi.org/10.1016/j.fmre.2024.04.011.

[15] Y. Akbari, N. Lei, N. Patel, Y. Peng, and O. Faust, "Atrial Fibrillation Detection on the Embedded Edge: Energy-Efficient Inference on a Low-Power Microcontroller," *Sensors*, vol. 25, no. 21, p. 6601, Oct. 2025, https://doi.org/10.3390/s25216601.

[16] M. Craighero, D. Quarantiello, B. Rossi, D. Carrera, P. Fragneto, and G. Boracchi, "On-Device Personalization for Human Activity Recognition on STM32," *IEEE Embedded Systems Letters*, vol. 16, no. 2, pp. 106–109, Jun. 2024, https://doi.org/10.1109/LES.2023.3293458.

[17] X. Wang, M. Hersche, M. Magno, and L. Benini, "MI-BMInet: An Efficient Convolutional Neural Network for Motor Imagery Brain–Machine Interfaces With EEG Channel Selection," *IEEE Sensors Journal*, vol. 24, no. 6, pp. 8835–8847, Mar. 2024, https://doi.org/10.1109/JSEN.2024.3353146.

[18] F. Pacini, T. Pacini, G. Lai, A. M. Zocco, and L. Fanucci, "Design and evaluation of CPU-, GPU-, and FPGA-based deployment of a CNN for motor imagery classification in brain-computer interfaces," *Electronics*, vol. 13, no. 9, p. 1646, 2024, https://doi.org/10.3390/electronics13091646.

[19] O. Sen, R. Soni, D. Virmani, A. Parekh, P. Lehman, S. Jena, A. Katikhaneni, A. Khalifa, and B. Chatterjee, "A low-latency neural inference framework for real-time handwriting recognition from EEG signals on an edge device," *Scientific Reports*, vol. 15, p. 41040, 2025, https://doi.org/10.1038/s41598-025-24972-y.

[20] J. W. Choi, J. Park, S. Huh, and S. Jo, "Asynchronous Motor Imagery BCI and LiDAR-Based Shared Control System for Intuitive Wheelchair Navigation," *IEEE Sensors Journal*, vol. 23, no. 14, pp. 16 252–16 263, Jul. 2023, https://doi.org/10.1109/JSEN.2023.3281756.

[21] X. Deng, Z. Liang Yu, C. Lin, Z. Gu, and Y. Li, "Self-adaptive shared control with brain state evaluation network for human-wheelchair cooperation," *Journal of Neural Engineering*, vol. 17, no. 4, p. 045005, Aug. 2020, https://doi.org/10.1088/1741-2552/ab937e.

[22] Q. Wei and X. Ding, "Intra- and Inter-Subject Common Spatial Pattern for Reducing Calibration Effort in MI-Based BCI," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 31, pp. 904–916, 2023, https://doi.org/10.1109/TNSRE.2023.3236372.

[23] D. Wu, J. Ouyang, N. Dai, M. Wu, H. Tan, H. Deng, Y. Fan, D. Wang, and Z. Jin, "DeepBrain: Enabling Fine-Grained Brain-Robot Interaction through Human-Centered Learning of Coarse EEG Signals from Low-Cost Devices," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 6, no. 3, pp. 1–27, Sep. 2022, https://doi.org/10.1145/3550334.

[24] P. Tarara, I. Przybył, J. Schöning, and A. Gunia, "Motor imagery-based brain-computer interfaces: An exploration of multiclass motor imagery-based control for Emotiv EPOC X," *Frontiers in Neuroinformatics*, vol. 19, p. 1625279, Aug. 2025, https://doi.org/10.3389/fninf.2025.1625279.

[25] B. Paneru, B. Thapa, B. Paneru, and S. C. Sapkota, "EEG right & left voluntary hand movement-based virtual brain-computer interfacing keyboard using hybrid deep learning approach," *Advanced Engineering Informatics*, vol. 65, p. 103304, May 2025, https://doi.org/10.1016/j.aei.2025.103304.

[26] J. R. Wolpaw, N. Birbaumer, D. J. McFarland, G. Pfurtscheller, and T. M. Vaughan, "Brain–computer interfaces for communication and control," *Clinical Neurophysiology*, vol. 113, no. 6, pp. 767–791, Jun. 2002, https://doi.org/10.1016/S1388-2457(02)00057-3.

[27] A. Al-Saegh, S. A. Dawwd, and J. M. Abdul-Jabbar, "Deep learning for motor imagery EEG-based classification: A review," *Biomedical Signal Processing and Control*, vol. 63, p. 102172, Jan. 2021, https://doi.org/10.1016/j.bspc.2020.102172.

[28] M. T. Sadiq, X. Yu, Z. Yuan, M. Z. Aziz, S. Siuly, and W. Ding, "Toward the Development of Versatile Brain–Computer Interfaces," *IEEE Transactions on Artificial Intelligence*, vol. 2, no. 4, pp. 314–328, Aug. 2021, https://doi.org/10.1109/TAI.2021.3097307.

[29] V. J. Lawhern, A. J. Solon, N. R. Waytowich, S. M. Gordon, C. P. Hung, and B. J. Lance, "EEGNet: A compact convolutional neural network for EEG-based brain–computer interfaces," *Journal of Neural Engineering*, vol. 15, no. 5, p. 056013, 2018, https://doi.org/10.1088/1741-2552/aace8c.

[30] T. M. Ingolfsson, M. Hersche, X. Wang, N. Kobayashi, L. Cavigelli, and L. Benini, "EEG-TCNet: An accurate temporal convolutional network for embedded motor-imagery brain–machine interfaces," in *2020 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 2020, pp. 2958–2965, https://doi.org/10.1109/SMC42975.2020.9283028.

[31] X. Deng, B. Zhang, N. Yu, K. Liu, and K. Sun, "Advanced TSGL-EEGNet for Motor Imagery EEG-Based Brain-Computer Interfaces," *IEEE Access*, vol. 9, pp. 25 118–25 130, 2021, https://doi.org/10.1109/ACCESS.2021.3056088.

[32] C. Wang, Y. Wu, C. Wang, Y. Zhu, C. Wang, Y. Niu, Z. Shao, X. Gao, Z. Zhao, and Y. Yu, "MI-EEG classification using Shannon complex wavelet and convolutional neural networks," *Applied Soft Computing*, vol. 130, p. 109685, Nov. 2022, https://doi.org/10.1016/j.asoc.2022.109685.

[33] J. Wang, L. Yao, and Y. Wang, "IFNet: An Interactive Frequency Convolutional Neural Network for Enhancing Motor Imagery Decoding From EEG," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 31, pp. 1900–1911, 2023, https://doi.org/10.1109/TNSRE.2023.3257319.

[34] R. T. Schirrmeister *et al.*, "Deep learning with convolutional neural networks for EEG decoding and visualization," *Human Brain Mapping*, vol. 38, no. 11, pp. 5391–5420, 2017, https://doi.org/10.1002/hbm.23730.

[35] E. Vafaei and M. Hosseini, "Transformers in EEG Analysis: A Review of Architectures and Applications in Motor Imagery, Seizure, and Emotion Classification," *Sensors*, vol. 25, no. 5, p. 1293, Feb. 2025, https://doi.org/10.3390/s25051293.

[36] W. Zhao, X. Jiang, B. Zhang, S. Xiao, and S. Weng, "CTNet: A convolutional transformer network for EEG-based motor imagery classification," *Scientific Reports*, vol. 14, no. 1, p. 20237, Aug. 2024, https://doi.org/10.1038/s41598-024-71118-7.

[37] Y. Ding, Y. Li, H. Sun, R. Liu, C. Tong, C. Liu, X. Zhou, and C. Guan, "EEG-Deformer: A Dense Convolutional Transformer for Brain-Computer Interfaces," *IEEE Journal of Biomedical and Health Informatics*, vol. 29, no. 3, pp. 1909–1918, Mar. 2025, https://doi.org/10.1109/JBHI.2024.3504604.

[38] H. Altaheri, F. Karray, and A.-H. Karimi, "Temporal convolutional transformer for EEG based motor imagery decoding," *Scientific Reports*, vol. 15, no. 1, p. 32959, Sep. 2025, https://doi.org/10.1038/s41598-025-16219-7.

[39] J. Wang, S. Cheng, J. Tian, and Y. Gao, "A 2D CNN-LSTM hybrid algorithm using time series segments of EEG data for motor imagery classification," *Biomedical Signal Processing and Control*, vol. 83, p. 104627, May 2023, https://doi.org/10.1016/j.bspc.2023.104627.

[40] G. Zoumpourlis and I. Patras, "Motor imagery decoding using ensemble curriculum learning and collaborative training," in *12th International Winter Conference on Brain-Computer Interface (BCI)*, 2024, pp. 1–8, https://doi.org/10.1109/BCI60775.2024.10480476.

[41] A. Keutayeva and B. Abibullaev, "Data Constraints and Performance Optimization for Transformer-Based Models in EEG-Based Brain-Computer Interfaces: A Survey," *IEEE Access*, vol. 12, pp. 62 628–62 647, 2024, https://doi.org/10.1109/ACCESS.2024.3394696.

[42] A. Keutayeva, N. Fakhrutdinov, and B. Abibullaev, "Compact convolutional transformer for subject-independent motor imagery EEG-based BCIs," *Scientific Reports*, vol. 14, no. 1, p. 25775, Oct. 2024, https://doi.org/10.1038/s41598-024-73755-4.

[43] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, arXiv:1503.02531. https://doi.org/10.48550/arXiv.1503.02531.

[44] Y. Li, E. Chen, X. Xiao, M. Xu, and D. Ming, "Lightweight deep learning models for EEG decoding: A review," *Journal of Neural Engineering*, vol. 22, no. 6, p. 061004, Dec. 2025, https://doi.org/10.1088/1741-2552/ae2717.

[45] Y. Zhou, Z. Chu, S. Cheng, M. Zhang, L. Chen, and Z. Wang, "From structural design to model compression: Research progress on lightweight algorithms for motor imagery brain–computer interfaces," *Journal of Intelligent Medicine*, 2026, https://doi.org/10.1002/jim4.70053.

[46] Y. Xie and S. Oniga, "A comprehensive review of hardware acceleration techniques and convolutional neural networks for EEG signals," *Sensors*, vol. 24, no. 17, p. 5813, 2024, https://doi.org/10.3390/s24175813.

[47] G. Huang, Z. Zhao, S. Zhang, Z. Hu, J. Fan, M. Fu, J. Chen, Y. Xiao, J. Wang, and G. Dan, "Discrepancy between inter- and intra-subject variability in EEG-based motor imagery brain-computer interface: Evidence from multiple perspectives," *Frontiers in Neuroscience*, vol. 17, p. 1122661, 2023, https://doi.org/10.3389/fnins.2023.1122661.

[48] D. Picard, "Torch.manual_seed(3407) is all you need: On the influence of random seeds in deep learning architectures for computer vision," 2021, arXiv:2109.08203. https://doi.org/10.48550/arXiv.2109.08203.

[49] X. Bouthillier, P. Delaunay, M. Bronzi, A. Trofimov, B. Nichyporuk, J. Szeto, N. M. Sepahvand, E. Raff, K. Madan, V. Voleti, S. Ebrahimi Kahou, V. Michalski, T. Arbel, C. Pal, G. Varoquaux, and P. Vincent, "Accounting for variance in machine learning benchmarks," in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 3, 2021, pp. 747–769, https://doi.org/10.48550/arXiv.2103.03098.

[50] F. Del Pup and M. Atzori, "Toward improving reproducibility in neuroimaging deep learning studies," *Frontiers in Neuroscience*, vol. 18, p. 1509358, 2024, https://doi.org/10.3389/fnins.2024.1509358.

[51] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," *International Journal of Computer Vision*, vol. 129, pp. 1789–1819, 2021, https://doi.org/10.1007/s11263-021-01453-z.

[52] A. M. Mansourian, R. Ahmadi, M. Ghafouri, A. M. Babaei, E. Badali Golezani, Z. Yasamani Ghamchi, V. Ramezanian, A. Taherian, K. Dinashi, A. Miri, and S. Kasaei, "A comprehensive survey on knowledge distillation," *Transactions on Machine Learning Research*, 2025, https://doi.org/10.48550/arXiv.2503.12067.

[53] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, "FitNets: Hints for thin deep nets," in *International Conference on Learning Representations (ICLR)*, 2015, https://doi.org/10.48550/arXiv.1412.6550.

[54] S. Javed, A. Mahmood, T. Qaiser, and N. Werghi, "Knowledge Distillation in Histology Landscape by Multi-Layer Features Supervision," *IEEE Journal of Biomedical and Health Informatics*, vol. 27, no. 4, pp. 2037–2046, Apr. 2023, https://doi.org/10.1109/JBHI.2023.3237749.

[55] W. Park, D. Kim, Y. Lu, and M. Cho, "Relational knowledge distillation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 3967–3976, https://doi.org/10.1109/CVPR.2019.00409.

[56] Y. Feng, X. Sun, W. Diao, J. Li, and X. Gao, "Double Similarity Distillation for Semantic Image Segmentation," *IEEE Transactions on Image Processing*, vol. 30, pp. 5363–5376, 2021, https://doi.org/10.1109/TIP.2021.3083113.

[57] J. Gou, X. Xin, B. Yu, H. Song, W. Zhang, and S. Wan, "Neighborhood relation-based knowledge distillation for image classification," *Neural Networks*, vol. 188, p. 107429, Aug. 2025, https://doi.org/10.1016/j.neunet.2025.107429.

[58] X. Ye, X. Fan, M. Zhang, R. Xu, and W. Zhong, "Unsupervised Monocular Depth Estimation via Recursive Stereo Distillation," *IEEE Transactions on Image Processing*, vol. 30, pp. 4492–4504, 2021, https://doi.org/10.1109/TIP.2021.3072215.

[59] L. Zhang, C. Bao, and K. Ma, "Self-distillation: Towards efficient and compact neural networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 8, 2022, https://doi.org/10.1109/TPAMI.2021.3067100.

[60] T. D. T. Mai, D.-Q. Vu, H. N. Phuong, and T.-N. Phung, "EEG-SKDNet: A self-knowledge distillation model with scaled weights for emotion recognition from EEG signals," *IEEE Access*, vol. 13, pp. 140 239–140 250, 2025, https://doi.org/10.1109/ACCESS.2025.3594671.

[61] Z. Wang, T. Gu, Y. Zhu, D. Li, H. Yang, and W. Du, "FLDNet: Frame-Level Distilling Neural Network for EEG Emotion Recognition," *IEEE Journal of Biomedical and Health Informatics*, vol. 25, no. 7, pp. 2533–2544, Jul. 2021, https://doi.org/10.1109/JBHI.2021.3049119.

[62] T. Gu, Z. Wang, X. Xu, D. Li, H. Yang, and W. Du, "Frame-Level Teacher–Student Learning With Data Privacy for EEG Emotion Recognition," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 12, pp. 11 021–11 028, Dec. 2023, https://doi.org/10.1109/TNNLS.2022.3168935.

[63] G. Zhang and A. Etemad, "Distilling EEG representations via capsules for affective computing," *Pattern Recognition Letters*, vol. 171, pp. 99–105, Jul. 2023, https://doi.org/10.1016/j.patrec.2023.05.011.

[64] Z. M. Ibrahim, D. Bean, T. Searle, L. Qian, H. Wu, A. Shek, Z. Kraljevic, J. Galloway, S. Norton, J. T. Teo, and R. J. Dobson, "A Knowledge Distillation Ensemble Framework for Predicting Short- and Long-Term Hospitalization Outcomes From Electronic Health Records Data," *IEEE Journal of Biomedical and Health Informatics*, vol. 26, no. 1, pp. 423–435, Jan. 2022, https://doi.org/10.1109/JBHI.2021.3089287.

[65] J. S. García-Salinas, A. A. Torres-García, C. A. Reyes-Garćia, and L. Villaseñor-Pineda, "Intra-subject class-incremental deep learning approach for EEG-based imagined speech recognition," *Biomedical Signal Processing and Control*, vol. 81, p. 104433, Mar. 2023, https://doi.org/10.1016/j.bspc.2022.104433.

[66] X.-Y. Huang, S.-Y. Chen, and C.-S. Wei, "Enhancing Low-Density EEG-Based Brain-Computer Interfacing With Similarity-Keeping Knowledge Distillation," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 8, no. 2, pp. 1156–1166, Apr. 2024, doi:10.1109/TETCI.2023.3335943. https://doi.org/10.1109/TETCI.2023.3335943.

[67] J. Wu, B. Tang, Y. Wang, C. Li, and Q. Yang, "A multi-level teacher assistant-based knowledge distillation framework with dynamic feedback for motor imagery EEG decoding," *Neural Networks*, vol. 194, p. 108180, Feb. 2026, doi:10.1016/j.neunet.2025.108180. https://doi.org/10.1016/j.neunet.2025.108180.

[68] Z. Xu and Z. Yu, "Entropy-based dual-teacher distillation for efficient motor imagery EEG classification," *Entropy*, vol. 28, no. 3, p. 310, 2026, https://doi.org/10.3390/e28030310.

[69] M. Schuld and N. Killoran, "Quantum Machine Learning in Feature Hilbert Spaces," *Physical Review Letters*, vol. 122, no. 4, p. 040504, Feb. 2019, https://doi.org/10.1103/PhysRevLett.122.040504.

[70] M. Schuld, A. Bocharov, K. M. Svore, and N. Wiebe, "Circuit-centric quantum classifiers," *Physical Review A*, vol. 101, no. 3, p. 032308, Mar. 2020, https://doi.org/10.1103/PhysRevA.101.032308.

[71] S. Jerbi, L. J. Fiderer, H. Poulsen Nautrup, J. M. Kübler, H. J. Briegel, and V. Dunjko, "Quantum machine learning beyond kernel methods," *Nature Communications*, vol. 14, no. 1, p. 517, 2023, https://doi.org/10.1038/s41467-023-36159-y.

[72] H.-Y. Huang, M. Broughton, M. Mohseni, R. Babbush, S. Boixo, H. Neven, and J. R. McClean, "Power of data in quantum machine learning," *Nature Communications*, vol. 12, no. 1, p. 2631, 2021, https://doi.org/10.1038/s41467-021-22539-9.

[73] R. K. Jha, N. Kasabov, S. Bhattacharyya, D. Coyle, and G. Prasad, "Comparative performance analysis of quantum feature maps for quantum kernel-based machine learning," *Scientific Reports*, vol. 16, p. 8142, 2026, https://doi.org/10.1038/s41598-026-39392-9.

[74] X.-D. Xie, X. Zhang, B. Koczor, and X. Yuan, "Advances in Quantum Computation in NISQ Era," *Entropy*, vol. 27, no. 10, p. 1074, Oct. 2025, https://doi.org/10.3390/e27101074.

[75] M. Alam and S. Ghosh, "QNet: A Scalable and Noise-Resilient Quantum Neural Network Architecture for Noisy Intermediate-Scale Quantum Computers," *Frontiers in Physics*, vol. 9, p. 755139, Jan. 2022, https://doi.org/10.3389/fphy.2021.755139.

[76] R. Idzikowski, M. A. Kucharski, K. Pempera, and M. Jaroszczuk, "A Survey on Quantum Machine Learning Applications in Medicine and Healthcare," *Applied Sciences*, vol. 16, no. 3, p. 1630, Feb. 2026, https://doi.org/10.3390/app16031630.

[77] C.-S. Chen, S. Y.-C. Chen, and H.-H. Tseng, "Exploring the potential of QEEGNet for cross-task and cross-dataset electroencephalography encoding with quantum machine learning," *Journal of Signal Processing Systems*, vol. 98, no. 1, p. 5, Jun. 2026, https://doi.org/10.1007/s11265-025-01979-2.

[78] C. Olvera, O. M. Ross, and Y. Rubio, "EEG-based motor imagery classification with quantum algorithms," *Expert Systems with Applications*, vol. 247, p. 123354, 2024, https://doi.org/10.1016/j.eswa.2024.123354.

[79] B. K. Behera, S. Al-Kuwari, and A. Farouk, "QSVM-QNN: Quantum support vector machine based quantum neural network learning algorithm for brain–computer interfacing systems," *IEEE Transactions on Artificial Intelligence*, vol. 7, pp. 308–320, Jan. 2026, https://doi.org/10.1109/TAI.2025.3572852.

[80] S. S. Dutta, I. M. Sawant, Sridevi S., G. S. Randhawa, R. Mistry, J. Ortega, Anandan P., and Indira B., "DuoQ-EpiNet: A dual-track quantum–classical convolutional neural network for EEG-based epilepsy seizure detection," *EPJ Quantum Technology*, vol. 13, no. 1, p. 37, Mar. 2026, https://doi.org/10.1140/epjqt/s40507-026-00500-3.

[81] Y. Wang, Y. Wang, C. Chen, R. Jiang, and W. Huang, "Development of variational quantum deep neural networks for image recognition," *Neurocomputing*, vol. 501, pp. 566–582, Aug. 2022, https://doi.org/10.1016/j.neucom.2022.06.010.

[82] I. Griol-Barres, S. Milla, A. Cebrián, Y. Mansoori, and J. Millet, "Variational Quantum Circuits for Machine Learning. An Application for the Detection of Weak Signals," *Applied Sciences*, vol. 11, no. 14, p. 6427, Jul. 2021, https://doi.org/10.3390/app11146427.

[83] J. Qi, C.-H. Yang, and P.-Y. Chen, "QTN-VQC: An end-to-end learning framework for quantum neural networks," *Physica Scripta*, vol. 99, no. 1, p. 015111, Jan. 2024, https://doi.org/10.1088/1402-4896/ad14d6.

[84] Y. Yan, S. Qian, Q. Zhao, and X. Zhang, "Distilling the knowledge with quantum neural networks," Mar. 2026, arXiv:2603.21586. https://doi.org/10.48550/arXiv.2603.21586.

[85] S. Oh, J. Kim, J. Park, H. Baek, H. Lee, J. Kim, and S.-L. Kim, "Quantum infidelity codistillation for fast and accurate distributed quantum machine learning," *The Journal of Supercomputing*, vol. 81, no. 10, p. 1151, Jul. 2025, https://doi.org/10.1007/s11227-025-07614-y.

[86] M. N. H. Nishan, M. J. Hasan, and M. R. C. Mahdy, "QuantumMedKD: A hybrid quantum–classical knowledge distillation framework for medical image analysis," *Alexandria Engineering Journal*, vol. 134, pp. 49–68, Jan. 2026, https://doi.org/10.1016/j.aej.2025.12.007.

[87] L. Barbato, M. Esposito, and F. Gargiulo, "Quantum-enhanced hybrid-model compression using knowledge distillation," *Quantum Machine Intelligence*, vol. 8, no. 2, p. 96, 2026, https://doi.org/10.1007/s42484-026-00431-3.

[88] Y. Yao, C. Cao, S. Haas, M. Agarwal, D. Khanna, and M. Abram, "Emulating quantum dynamics with neural networks via knowledge distillation," *Frontiers in Materials*, vol. 9, p. 1060744, Jan. 2023, https://doi.org/10.3389/fmats.2022.1060744.

[89] C.-Y. Liu, K.-C. Chen, K. Murota, S. Y.-C. Chen, and E. Rinaldi, "Quantum relational knowledge distillation," 2025, arXiv:2508.13054. https://doi.org/10.48550/arXiv.2508.13054.

[90] C.-Y. Liu, E.-J. Kuo, C.-H. Abraham Lin, J. Gemsun Young, Y.-J. Chang, M.-H. Hsieh, and H.-S. Goan, "Quantum-Train: Rethinking hybrid quantum-classical machine learning in the model compression perspective," *Quantum Machine Intelligence*, vol. 7, no. 2, p. 80, Dec. 2025, doi:10.1007/s42484-025-00305-0. https://doi.org/10.1007/s42484-025-00305-0.

[91] J. Liu, F. Tacchino, J. R. Glick, L. Jiang, and A. Mezzacapo, "Representation Learning via Quantum Neural Tangent Kernels," *PRX Quantum*, vol. 3, no. 3, p. 030323, Aug. 2022, https://doi.org/10.1103/PRXQuantum.3.030323.

[92] A. Pérez-Salinas, A. Cervera-Lierta, E. Gil-Fuster, and J. I. Latorre, "Data re-uploading for a universal quantum classifier," *Quantum*, vol. 4, p. 226, Feb. 2020, https://doi.org/10.22331/q-2020-02-06-226.

[93] S. Thanasilp, S. Wang, M. Cerezo, and Z. Holmes, "Exponential concentration in quantum kernel methods," *Nature Communications*, vol. 15, no. 1, p. 5200, 2024, https://doi.org/10.1038/s41467-024-49287-w.

[94] R. Shaydulin and S. M. Wild, "Importance of kernel bandwidth in quantum machine learning," *Physical Review A*, vol. 106, no. 4, p. 042407, 2022, https://doi.org/10.1103/physreva.106.042407.

[95] G. Schalk, D. J. McFarland, T. Hinterberger, N. Birbaumer, and J. R. Wolpaw, "BCI2000: A general-purpose brain-computer interface (BCI) system," *IEEE Transactions on Biomedical Engineering*, vol. 51, no. 6, pp. 1034–1043, 2004, https://doi.org/10.1109/TBME.2004.827072.

[96] A. L. Goldberger *et al.*, "PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000, https://doi.org/10.1161/01.CIR.101.23.e215.

[97] Y. Xu, X. Huang, and Q. Lan, "Selective cross-subject transfer learning based on Riemannian tangent space for motor imagery brain-computer interface," *Frontiers in Neuroscience*, vol. 15, p. 779231, 2021, https://doi.org/10.3389/fnins.2021.779231.

[98] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023, https://doi.org/10.1016/j.patter.2023.100804.

[99] P. Yuan, X. Gao, B. Allison, Y. Wang, G. Bin, and S. Gao, "A study of the existing problems of estimating the information transfer rate in online brain–computer interfaces," *Journal of Neural Engineering*, vol. 10, no. 2, p. 026014, Apr. 2013, https://doi.org/10.1088/1741-2560/10/2/026014.

[100] H. Cho, M. Ahn, S. Ahn, M. Kwon, and S. C. Jun, "EEG datasets for motor imagery brain-computer interface," *GigaScience*, vol. 6, no. 7, p. gix034, 2017, https://doi.org/10.1093/gigascience/gix034.

[101] P. Sedi Nzakuna, E. D'Auria, V. Paciello, V. Gallo, E. N. Kamavuako, A. Lay-Ekuakille, and K. Kyamakya, "Real-world evaluation of deep learning decoders for motor imagery EEG-based BCIs," *Frontiers in Systems Neuroscience*, vol. 19, p. 1718390, 2025, https://doi.org/10.3389/fnsys.2025.1718390.